AI IN ACTION - Day 1

AI & LLM Foundation

Bạn đang dùng AI mỗi ngày — nhưng thực sự bên trong nó đang làm gì?

Instructor: Mai Anh Nguyen (Blue)

Instructor

Mai Anh Nguyen (Blue)

Generalist Product Builder

- 2026 FPT Long Châu (PM · Healthcare Product)
- 2025 Thongtincuuho.org (Co-founder)
- 2025 FPT Software AI Center (PM · AI Agent)
- 2021 - 2025 Xantus (PM · On-chain Analytics, AI Agent)
- 2016 - 2021 DYNO, Kalapa (PM · OCR, eKYC, Credit Scoring)

Linkedin | Facebook

Agenda

- Bức tranh AI & các tầng của AI
- Lịch sử AI 70 năm
- Bên trong LLM: cơ chế vận hành
- Từ LLM đến AI Agent
- Landscape: model hôm nay & cuộc đua hiện tại
- Chọn model & chi phí token
- Gọi API lần đầu
- Tổng kết — những ý để mang về

AI & LLM Foundation

Từ "nghe AI" đến "gọi AI" trong một ngày

Hôm nay mình đi từ "nghe AI" đến "gọi AI"

Cuối ngày này, mỗi bạn sẽ ra về với 4 thứ:

1 Hiểu được

Giải thích được LLM hoạt động thế nào — bằng trực giác, không cần công thức

2 Nắm được

Token, context, chi phí, độ trễ liên hệ với nhau ra sao

3 Gọi được

Lần gọi API đầu tiên — và hiểu cấu trúc của một lần gọi model

4 Build được

Một chatbot dòng lệnh đơn giản có streaming — sản phẩm của chính bạn

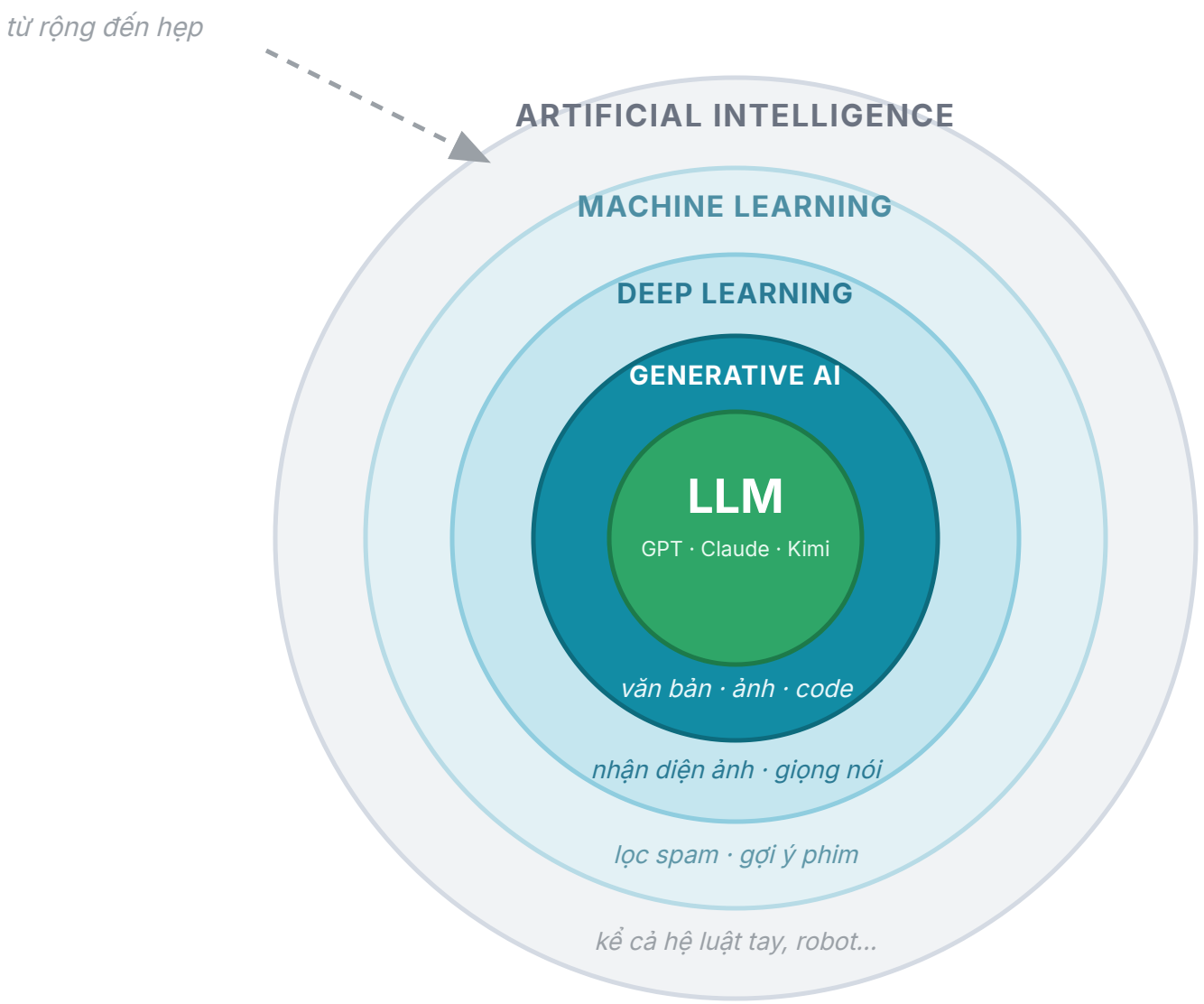
Không cần nền toán. Chỉ cần tò mò và một chiếc máy tính.

PHẦN 01

Bức tranh AI

AI, machine learning, LLM nằm ở đâu trong cùng một hệ?

AI, ML, Deep Learning, GenAI, LLM — nằm ở đâu trong cùng một hệ?



- **AI** — chiếc ô lớn nhất: mọi hệ thống có yếu tố "thông minh".
- **Machine learning** — học từ dữ liệu thay vì viết luật tay.
- **Deep learning** — mạng nơ-ron nhiều tầng tự học đặc trưng.
- **Generative AI** — sinh nội dung mới: văn bản, ảnh, code.
- **LLM** — model nền chuyên ngôn ngữ, tim của làn sóng hiện nay.

LLM không phải toàn bộ AI — nhưng nó là tầng nền của gần hết trải nghiệm AI bạn dùng hôm nay

Ba nhóm AI chính: phân loại · sinh nội dung · hành động

Discriminative AI

Giỏi **phân loại, dự đoán**: lọc spam, phát hiện gian lận, nhận diện ảnh.

Input → một nhãn, một con số

Generative AI

Sinh ra thứ mới: văn bản, ảnh, code. ChatGPT, Claude, Midjourney.

Prompt → nội dung mới

Agentic AI

Nhận **mục tiêu** rồi tự làm nhiều bước: lập kế hoạch, dùng công cụ, hành động.

Goal → Plan → Action

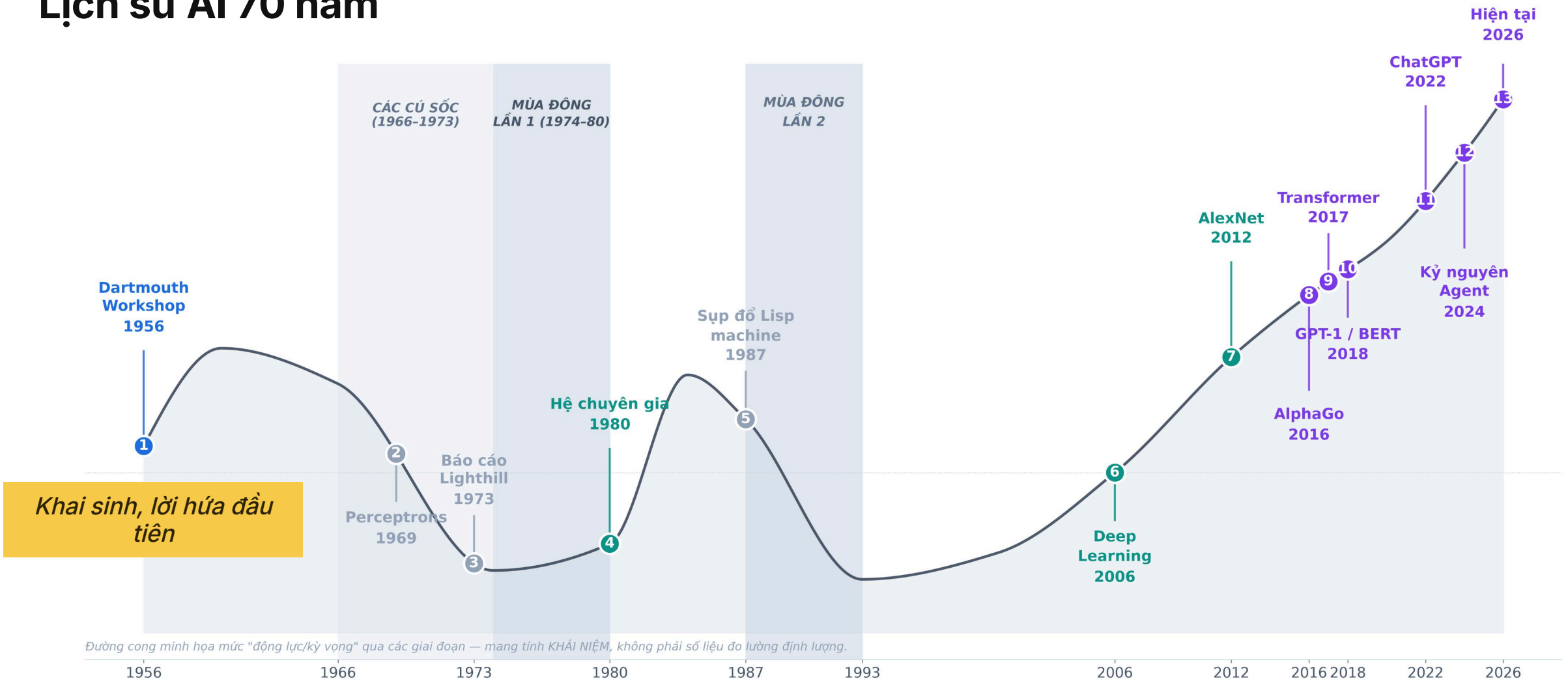
*LLM là **engine chung** của cả Generative lẫn Agentic — cuối buổi sáng mình sẽ thấy agent khác LLM ở đâu*

PHẦN 02

Lịch sử AI

70 năm của những lần chạm trần và đổi nền tảng

Lịch sử AI 70 năm

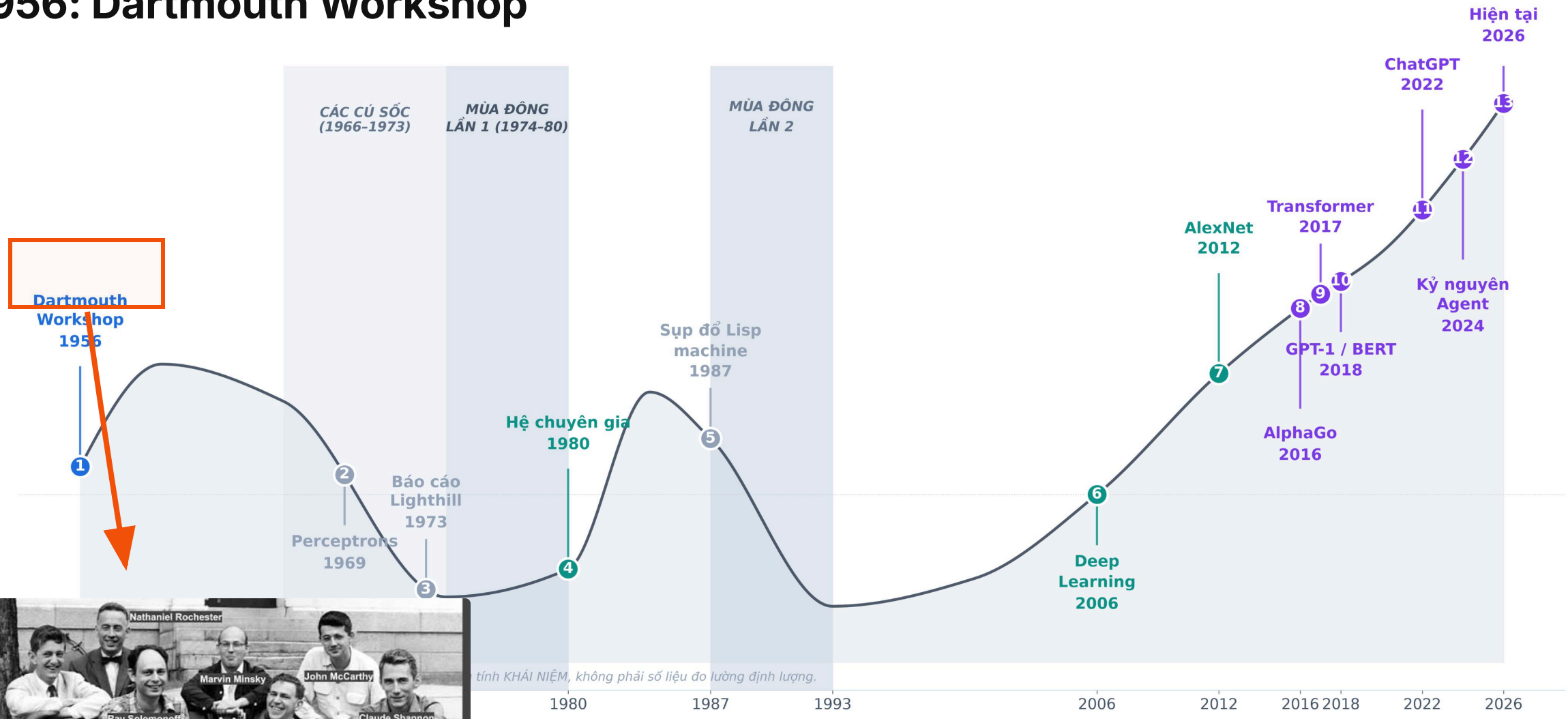


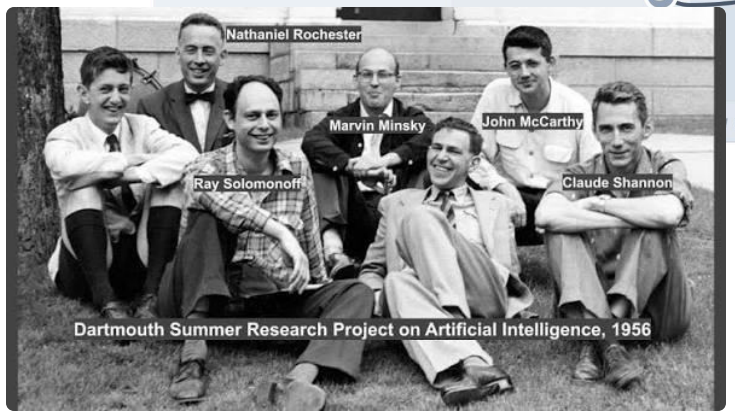
Khai sinh, lời hứa đầu tiên

2 lần mùa đông, cách tiếp cận chạm trần

Từ model đơn lẻ sang system có khả năng hành động như agent

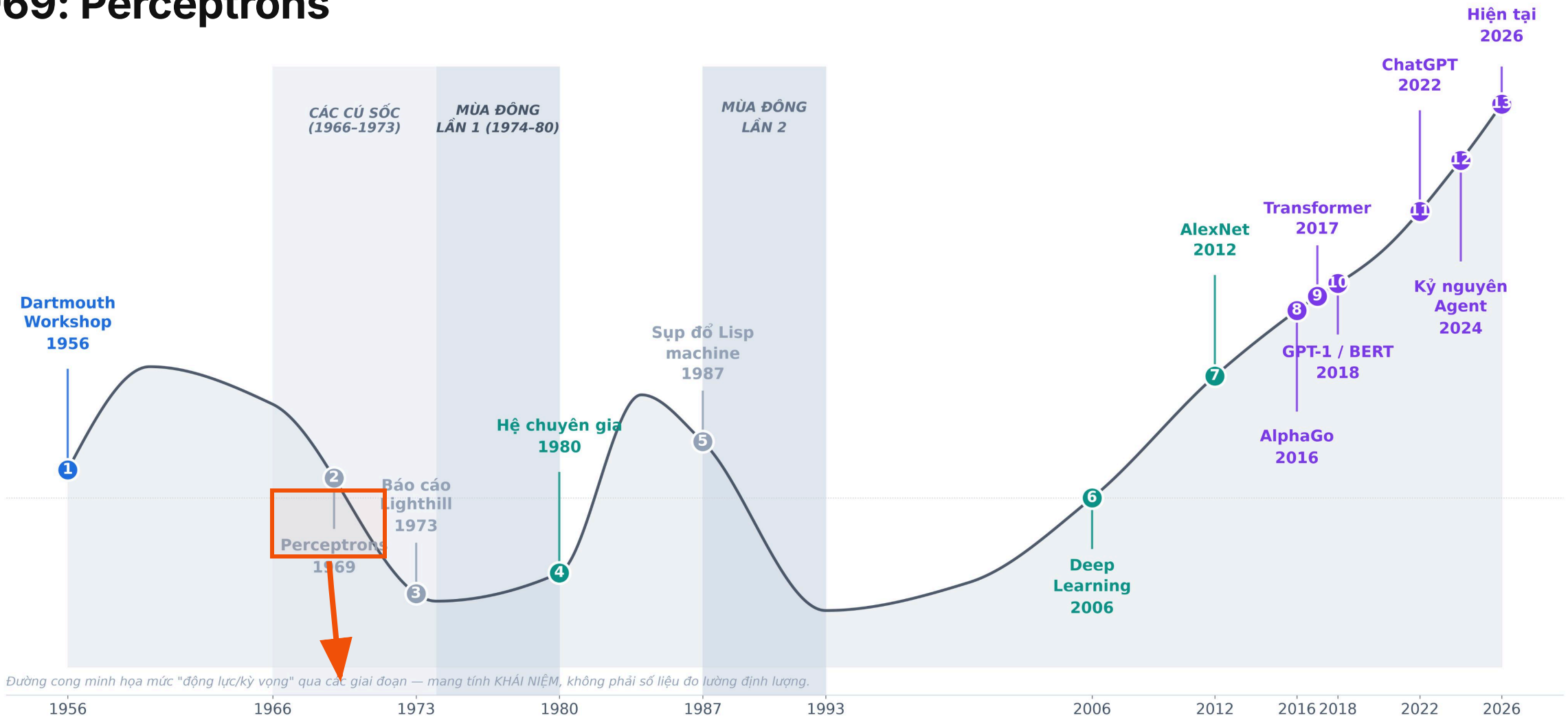
1956: Dartmouth Workshop





"Artificial Intelligence" ra đời với ý tưởng: nếu trí thông minh có thể được mô tả đủ rõ, thì máy móc cũng có thể mô phỏng lại nó.

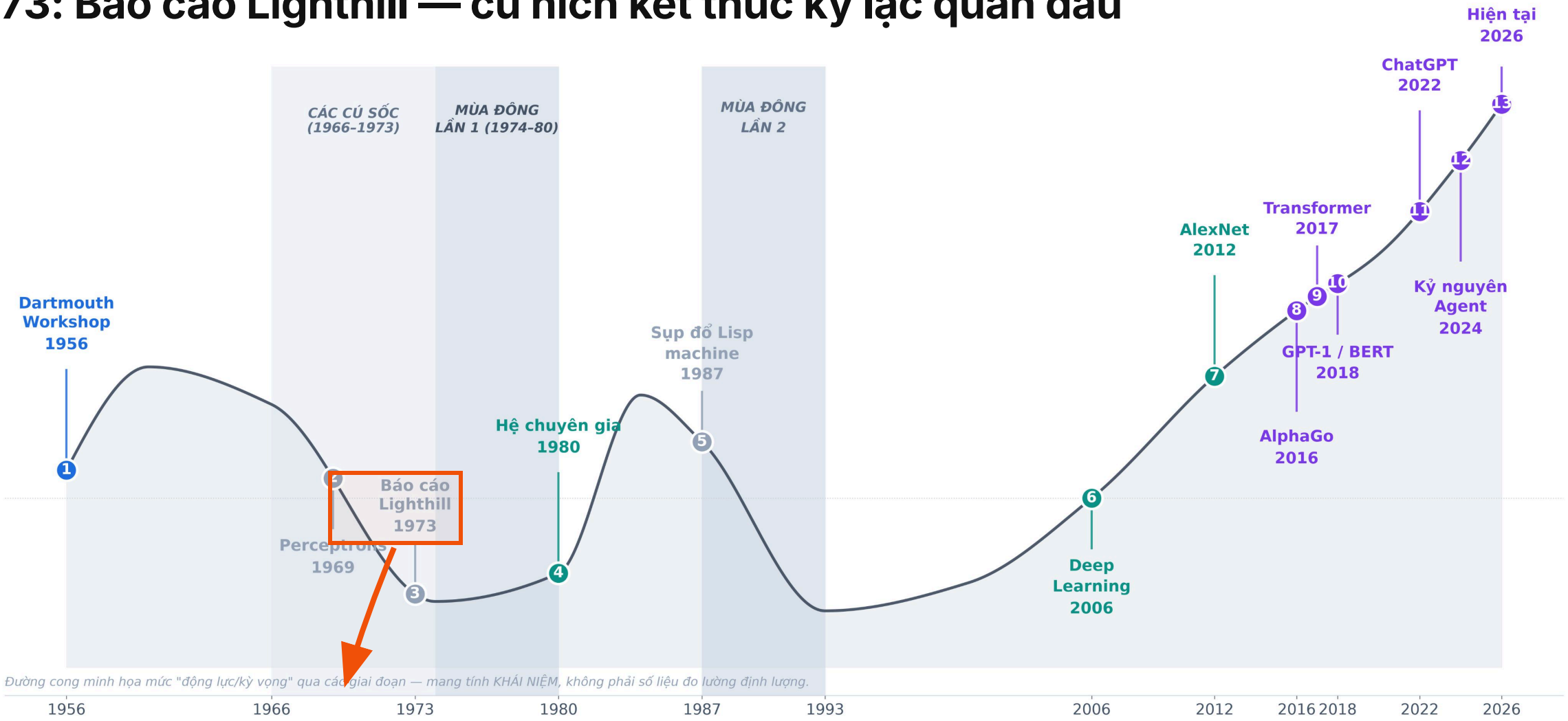
1969: Perceptrons



Các hướng đi lần lượt **chạm trần**:

- **Hướng symbolic** (dạy máy bằng luật/quy tắc): bắt đầu đuối trước thế giới quá nhiều ngữ cảnh
- **Hướng Perceptron** (thay vì viết hết luật, mình có thể cho máy học từ ví dụ) cũng gặp vấn đề vì quá đơn giản

1973: Báo cáo Lighthill — cú hích kết thúc kỳ lạc quan đầu



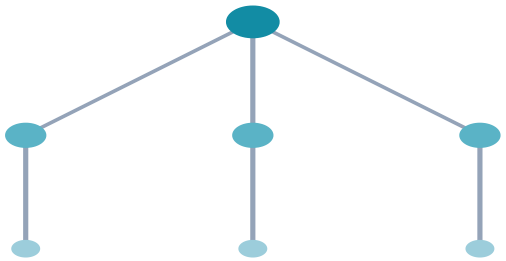
Chính phủ Anh nhờ James Lighthill đánh giá lại toàn ngành AI. Ông kết luận thẳng: **những gì AI làm được đi quá xa so với lời hứa**.

Nguồn tiền đổ vào AI ở Anh và Mỹ bị cắt mạnh → mở màn **mùa đông AI lần thứ nhất**.

Lighthill, J. (1973), "Artificial Intelligence: A General Survey", Science Research Council — chilton-computing.org.uk

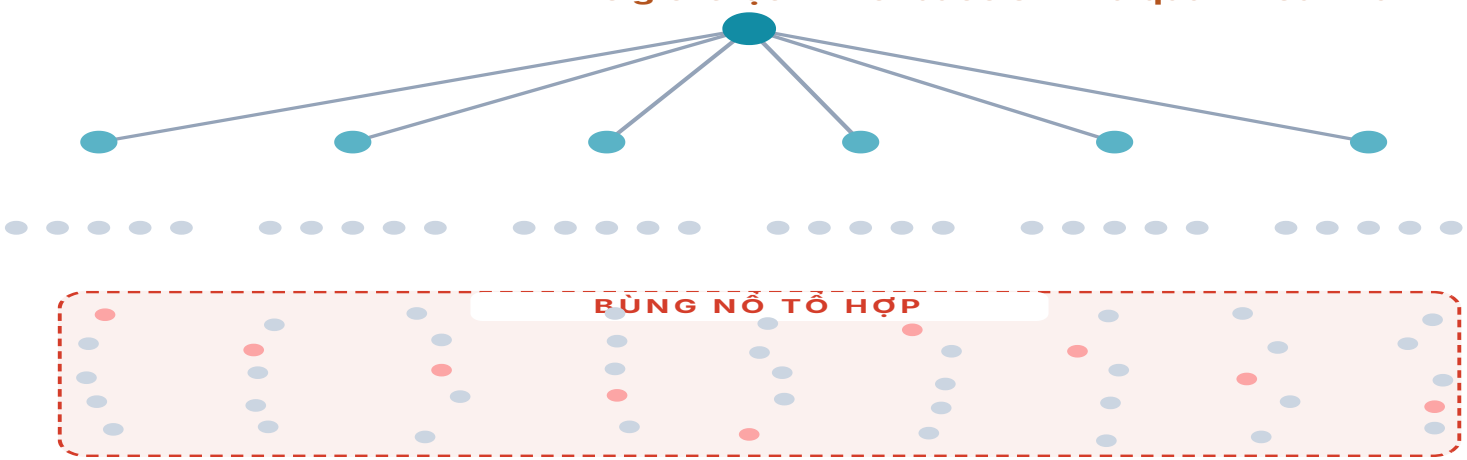
Mùa đông AI lần 1: 1974-1980

Bài toán nhỏ — trông khá thông minh ✓

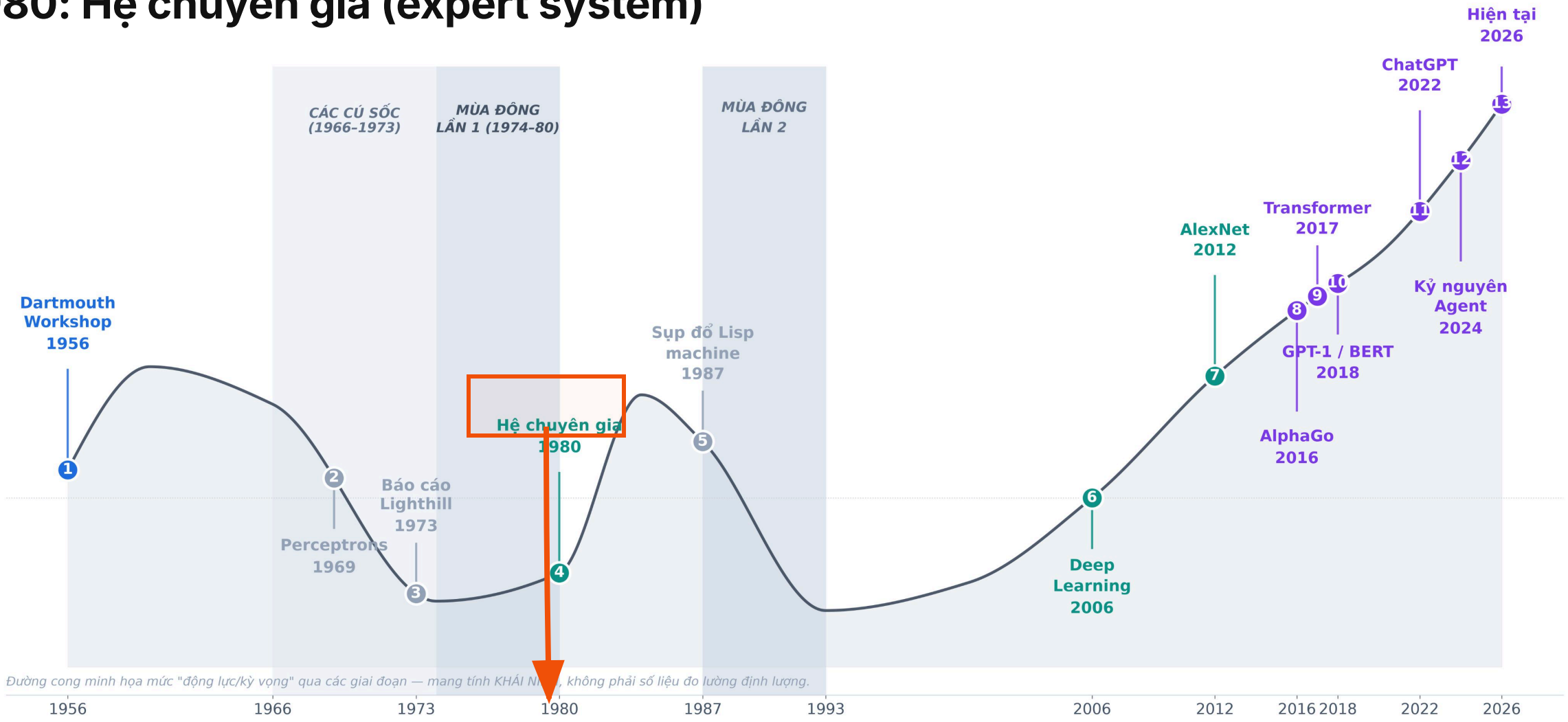


Ít nhánh, máy duyệt hết được → kết quả trông "thông minh".

Thế giới thật — mỗi bước sinh ra quá nhiều nhánh



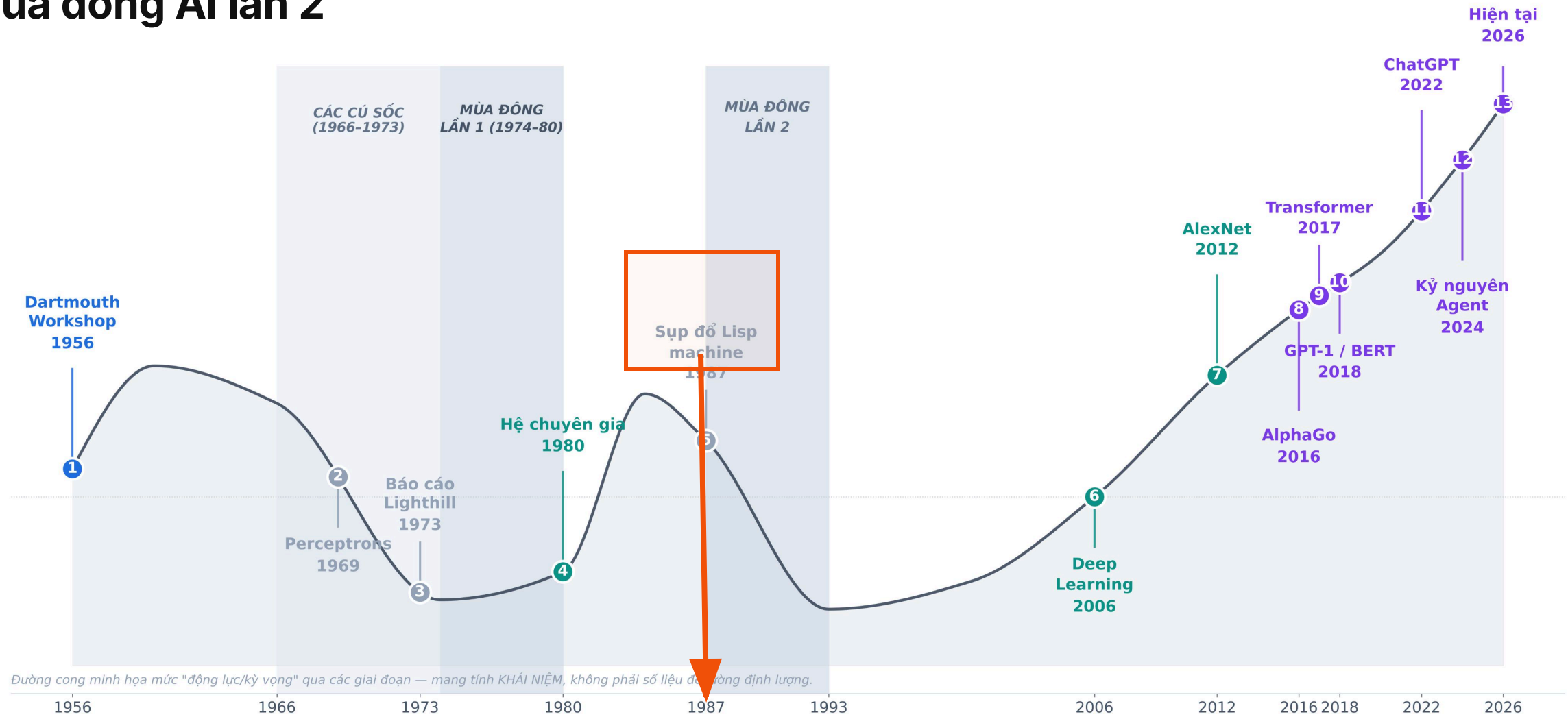
1980: Hệ chuyên gia (expert system)



Đặt lại vấn đề: *"Nếu AI chỉ giải thật tốt một loại bài toán chuyên môn hẹp thì sao?"*
→ Sự ra đời của **expert systems**

AI đổi chiến lược: thôi theo đuổi trí tuệ tổng quát và **tập trung giải thật tốt một miền hẹp** bằng cách mã hóa tri thức chuyên gia thành luật

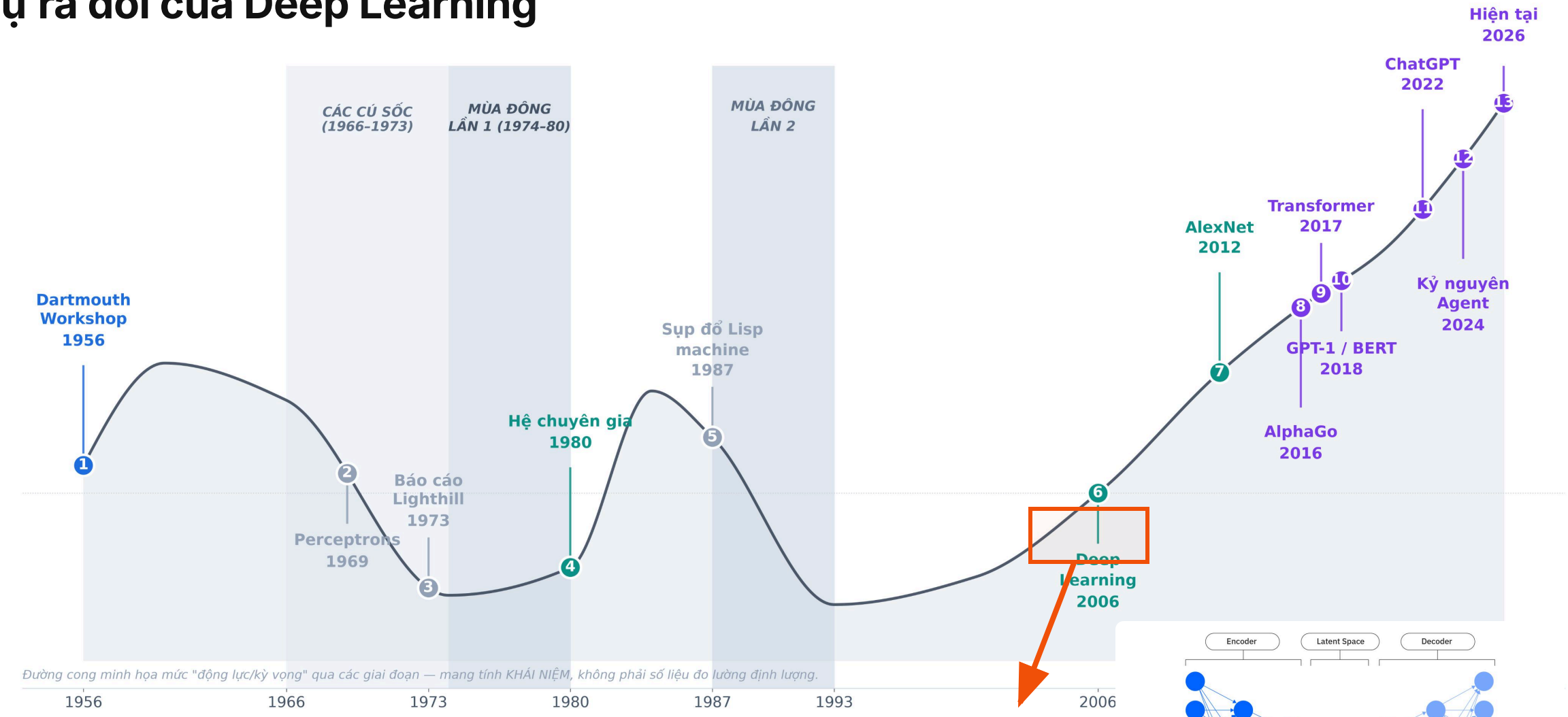
Mùa đông AI lần 2

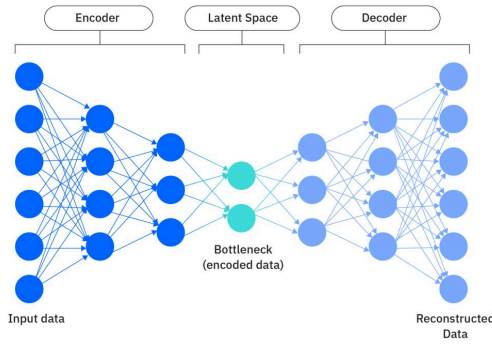


Expert systems từng tạo ra giá trị thật, nhưng càng mở rộng thì càng lộ trần: tri thức phải nhập bằng tay, luật càng nhiều càng khó cập nhật, và hệ thống khó đứng vững trước ngoại lệ mới.

→ Mùa đông AI lần 2

Sự ra đời của Deep Learning

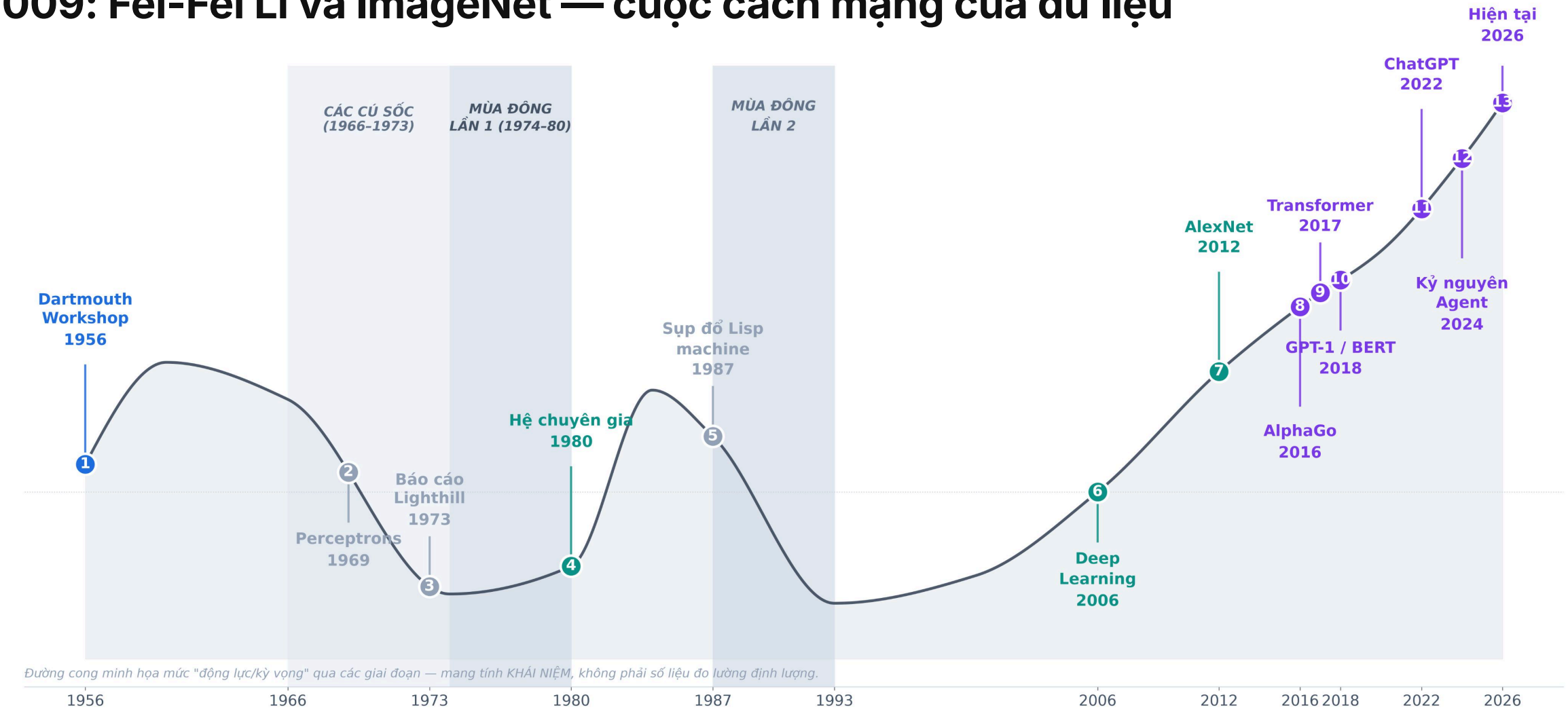




Sau mùa đông lần hai, câu hỏi của cả ngành đổi hẳn:

"Nếu không thể viết hết tri thức thế giới vào máy, thì có thể để máy tự học nó từ dữ liệu không?"

2009: Fei-Fei Li và ImageNet — cuộc cách mạng của dữ liệu

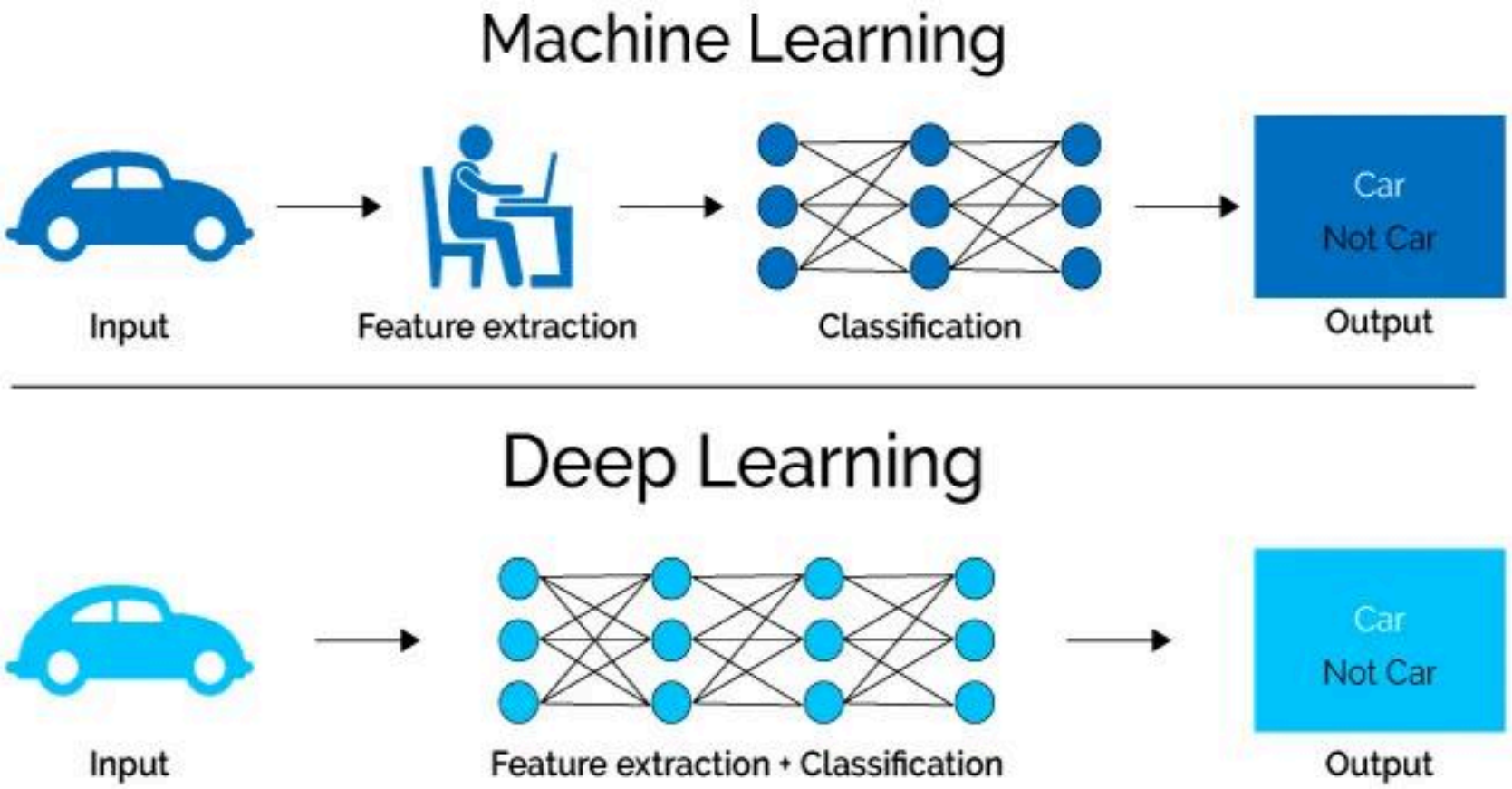


Trong khi cả ngành chạy theo thuật toán thông minh hơn, Fei-Fei Li chọn con đường khác: **xây bộ dữ liệu lớn hơn** — 14 triệu ảnh được gán nhãn tay, hơn 20.000 loại vật.

Ba năm sau, chính bộ dữ liệu đó là sân khấu cho cú nổ AlexNet 2012 → bài học định hình cả kỷ nguyên: **đôi khi dữ liệu tốt hơn đánh bại thuật toán khôn hơn**.

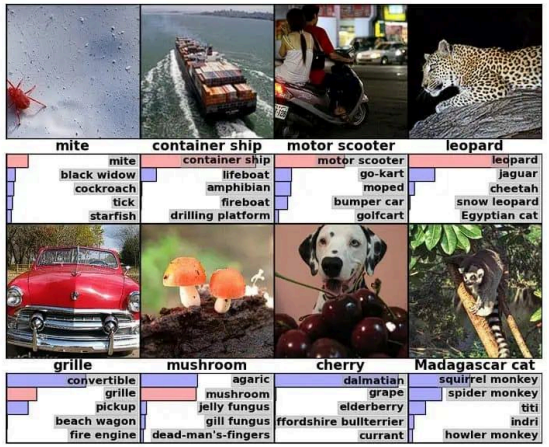
Deng, J. et al. (2009), "ImageNet: A Large-Scale Hierarchical Image Database", CVPR — doi.org/10.1109/CVPR.2009.5206848 · Fei-Fei Li, TED 2015 — ted.com

Deep Learning khác Machine Learning truyền thống ở chỗ nào?

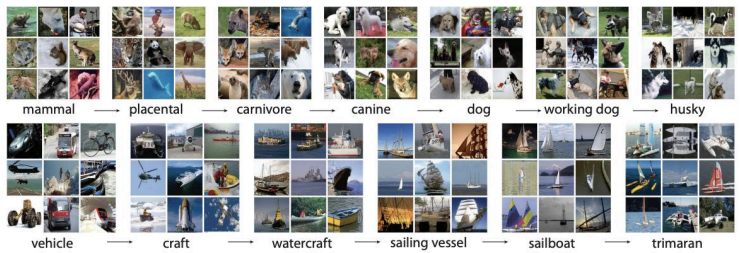


Không cần con người thiết kế đặc trưng bằng tay — mạng sâu TỰ học đặc trưng từ dữ liệu thô, từ đơn giản đến phức tạp

2012: AlexNet



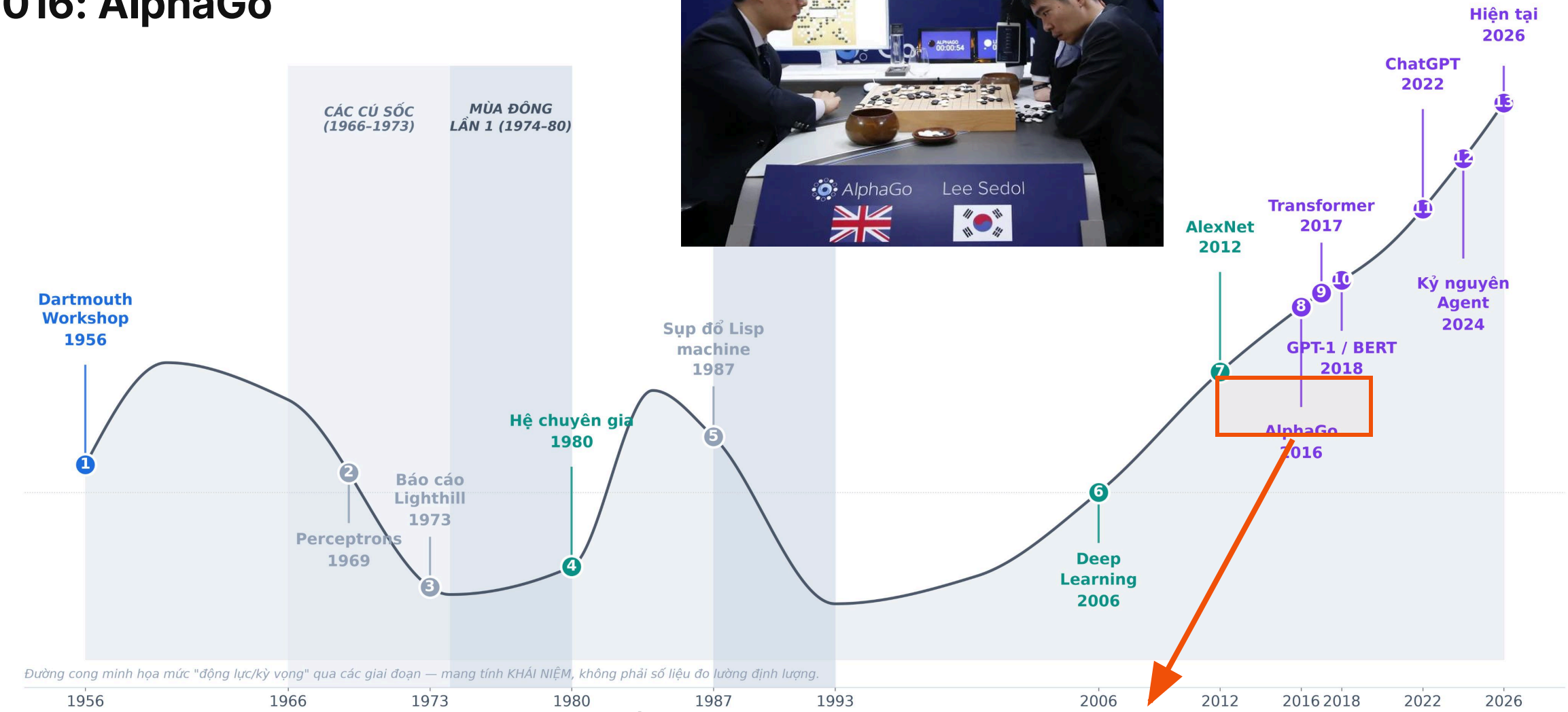
Imag



AlexNet chiến thắng ở ImageNet Large Scale Visual Recognition Challenge

- **ImageNet** cho mô hình ăn một lượng dữ liệu chưa từng có ở thời điểm đó.
- **Kiến trúc sâu** cho phép học dần từ cạnh, hình, bộ phận, rồi đến đối tượng.
- **GPU** cung cấp đủ năng lực tính toán để quá trình huấn luyện trở nên khả thi.

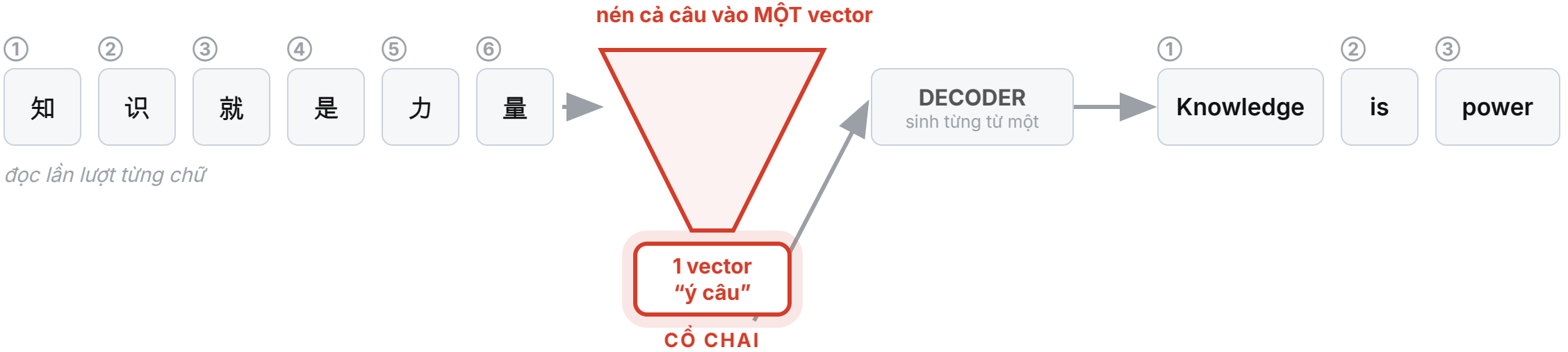
2016: AlphaGo



AlphaGo và nước đi số 37

Ban đầu nó học từ khoảng 150.000 ván cờ của chuyên gia con người để có trực giác khởi đầu
→ Tạo ra nhiều bản sao của AlphaGo và để chúng **tự chơi với chính mình** hàng triệu lần
→ Hệ thống không chỉ học từ những gì con người đã biết, mà còn **tự mở rộng không gian chiến lược** bằng cách **khám phá những nước đi chưa từng được thử** trước đó.

Nút thắt của RNN: đọc hết rồi mới nói — từng bước một



① Câu càng dài → càng quên chữ đầu

Hôm qua tôi đi chợ mua được một con **cá to**

chữ đầu "mờ" dần trong vector duy nhất — như người cố nhớ một câu rất dài bằng trí nhớ ngắn hạn

② Từng bước một → chậm, khó mở rộng

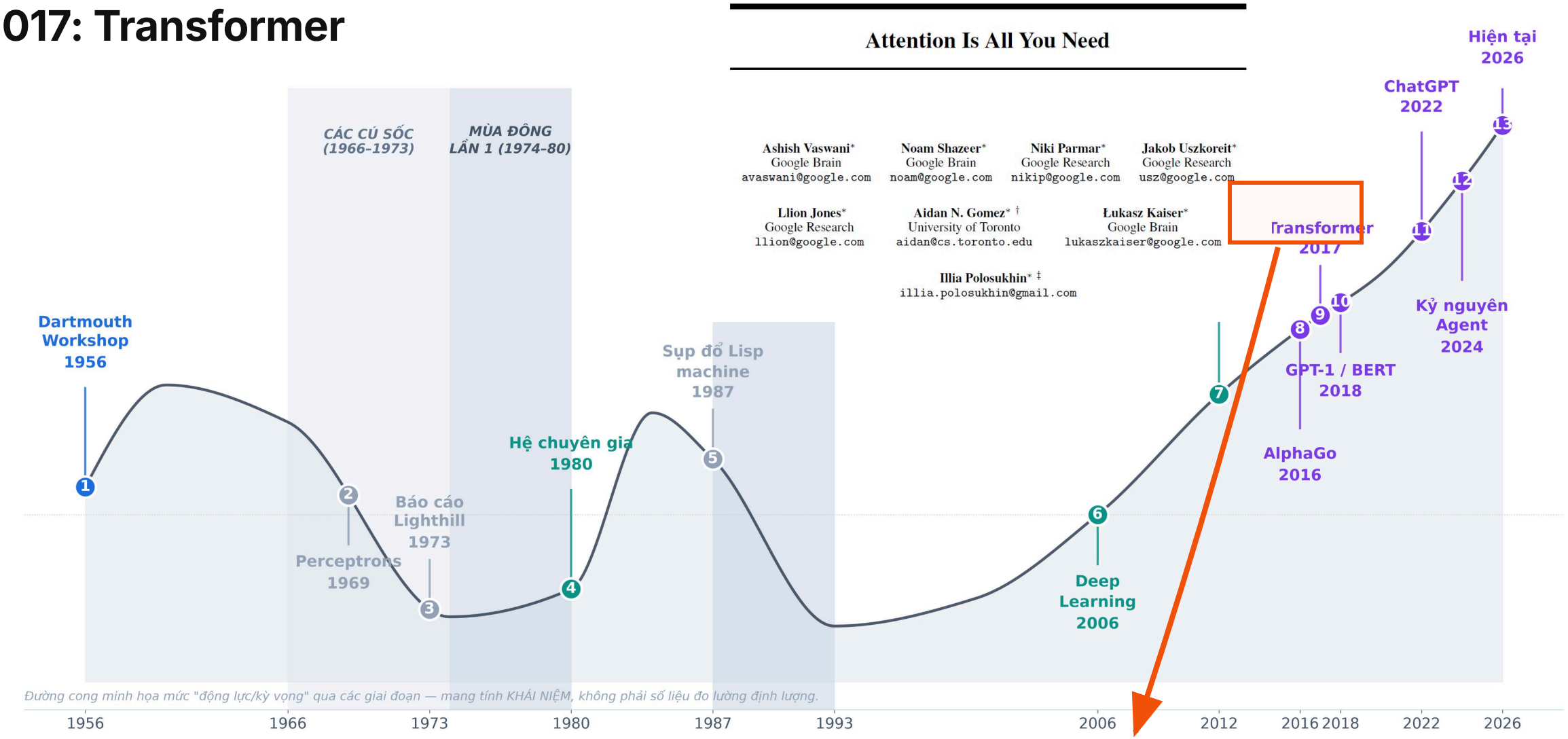
1 → 2 → 3 → … → 100 ⏱

*muốn chữ thứ 100 phải chờ đủ 99 bước trước — **không song song được**, khó scale lên model lớn*

Transformer thắng không phải vì phép màu — nó tháo đúng nút thắt này: cho mọi từ nhìn nhau cùng lúc

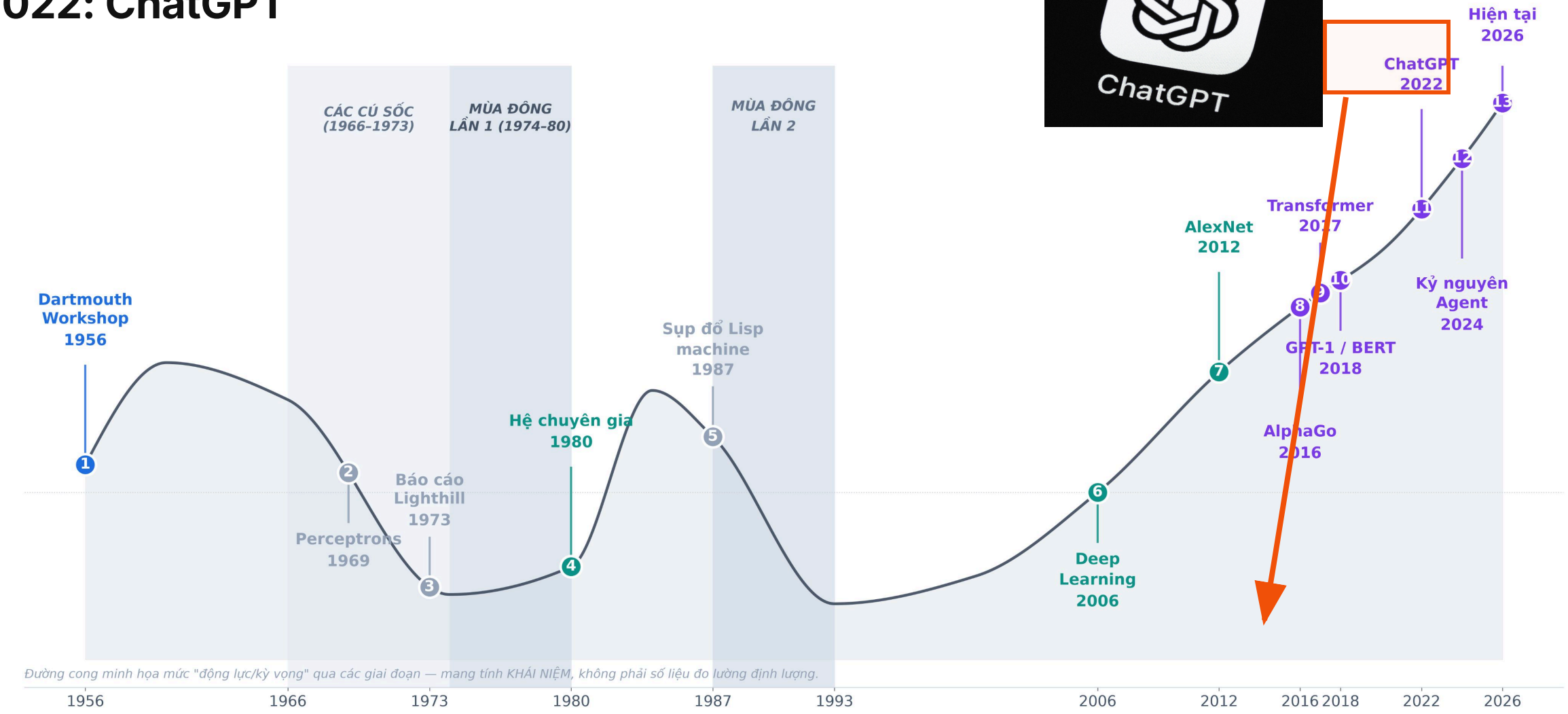
Sutskever et al. (2014), "Sequence to Sequence Learning with Neural Networks" · Wu et al. (2016), Google Neural Machine Translation — arxiv.org/abs/1609.08144

2017: Transformer



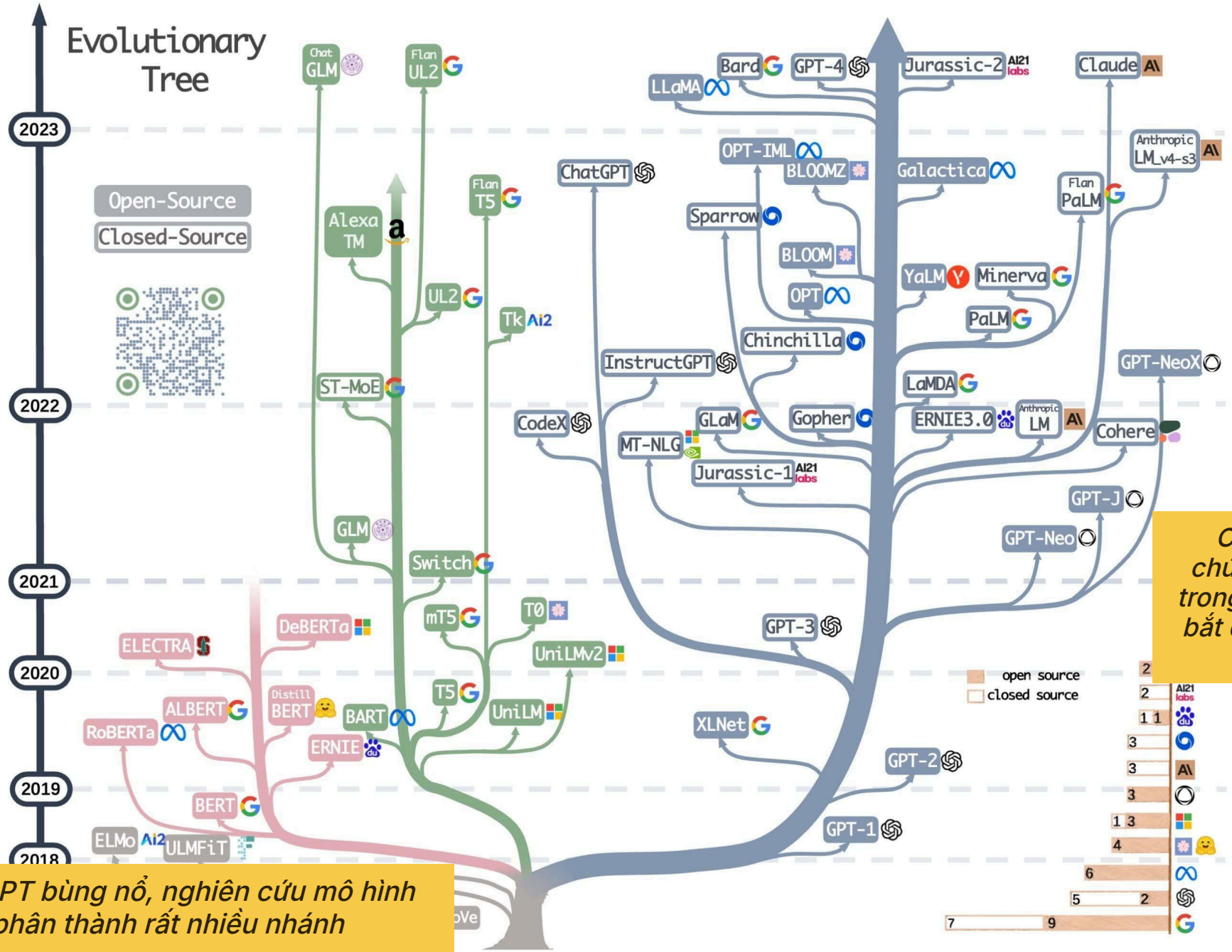
Transformer là bước ngoặt vì nó cho mô hình hiểu ngôn ngữ theo cách linh hoạt hơn: **mỗi từ có thể nhìn sang những từ quan trọng khác** trong cả câu, thay vì chỉ đi tuần tự từng bước → trở thành nền móng kỹ thuật cho GPT, BERT và toàn bộ làn sóng LLM sau đó.

2022: ChatGPT



ChatGPT xuất hiện như một trải nghiệm đại chúng

Lần đầu tiên rất đông người dùng phổ thông có thể trực tiếp chạm vào một mô hình ngôn ngữ mạnh, thông qua một giao diện đơn giản đến mức ai cũng hiểu cách dùng



Trước khi ChatGPT bùng nổ, nghiên cứu mô hình ngôn ngữ phân thành rất nhiều nhánh

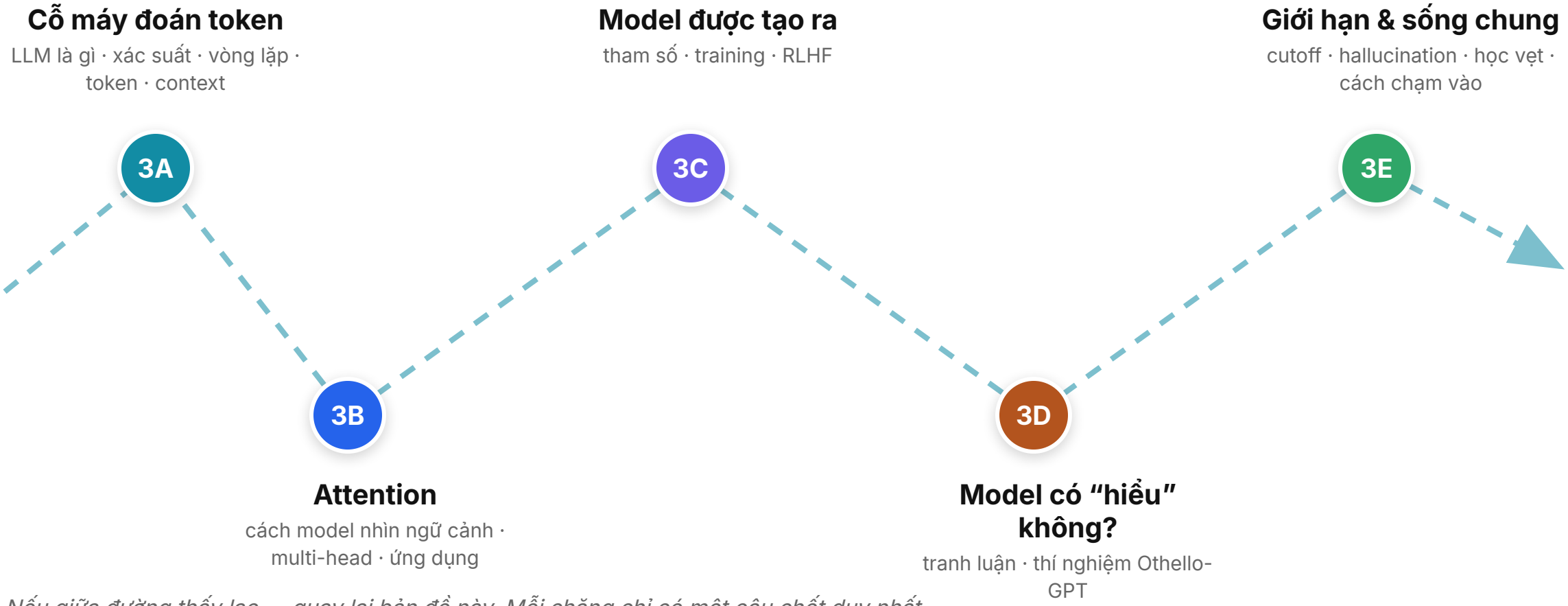
ChatGPT xuất hiện, chứng minh hiệu quả → trong tâm của toàn ngành bắt đầu dồn về cùng một trục

PHẦN 03

Bên trong LLM

từ vòng lặp đoán token đến giới hạn của model

Bên trong LLM — bản đồ 5 chặng của buổi sáng



Nếu giữa đường thấy lạc — quay lại bản đồ này. Mỗi chặng chỉ có một câu chốt duy nhất.

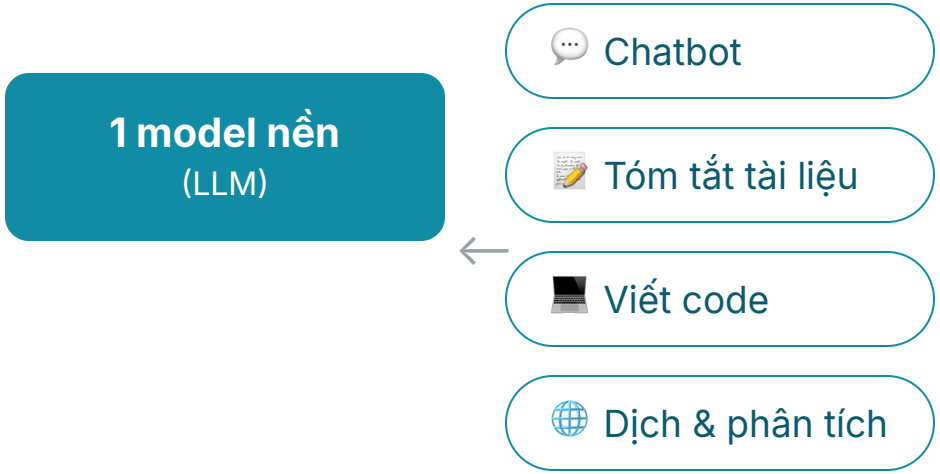
*Thần chú xuyên suốt: **"Model chỉ đoán token tiếp theo — mọi thứ khác là hệ quả."***

LLM là gì? — một bộ não nền, không phải một chatbot

LLM (Large Language Model) là một mô hình ngôn ngữ rất lớn, thường dựa trên kiến trúc Transformer, được luyện trên hàng nghìn tỷ mảnh chữ để học cách **đoán mảnh chữ tiếp theo trong ngữ cảnh**.

Nhờ được luyện đủ rộng, nó trở thành một **nền chung**: thay vì mỗi việc train một model riêng, cùng một model làm được rất nhiều việc.

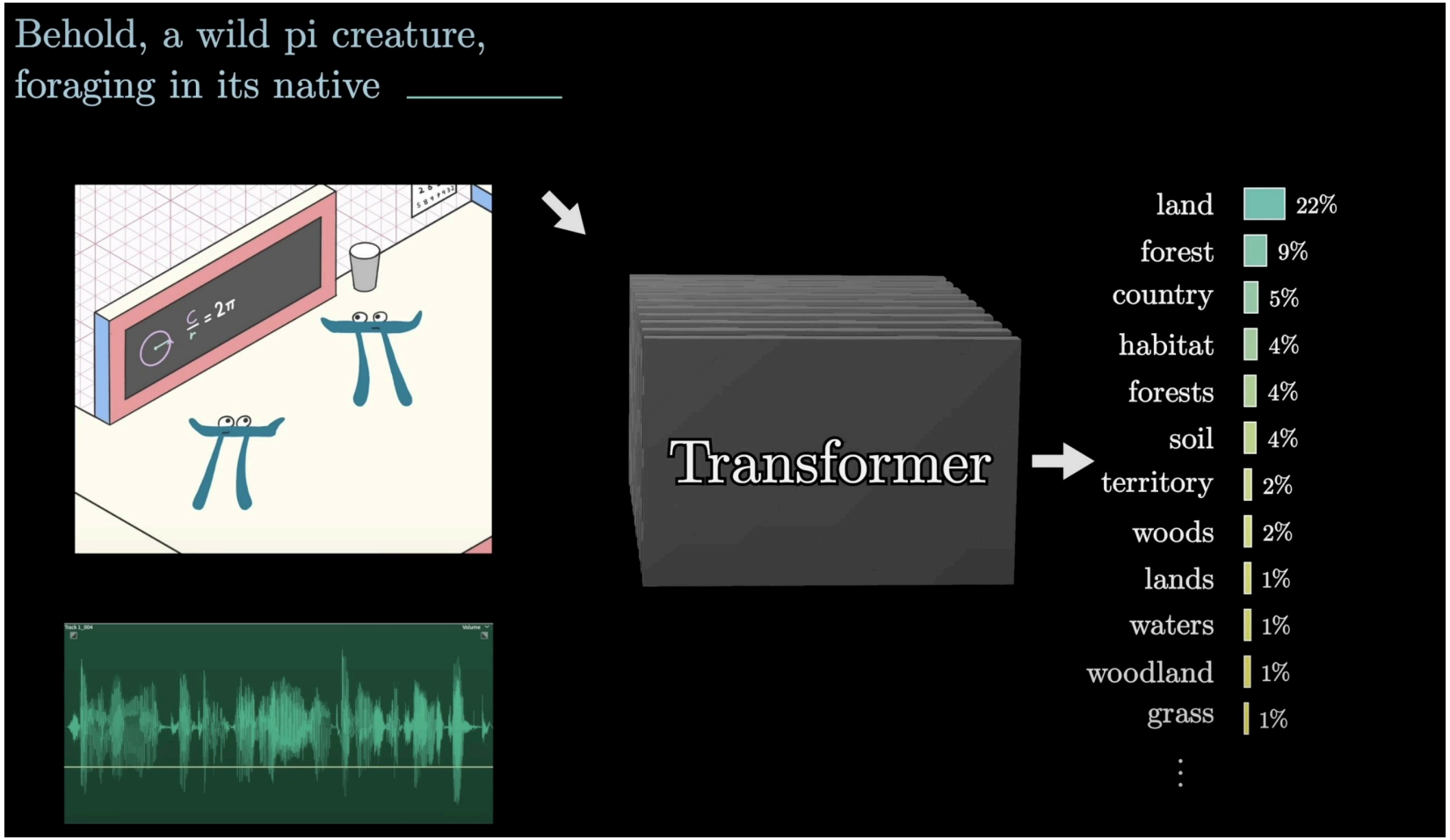
Chatbot chỉ là **một dạng sản phẩm** đóng gói quanh bộ não đó — lớp áo bên ngoài.



LLM = bộ não ngôn ngữ dùng chung cho mọi việc — sản phẩm bạn thấy chỉ là lớp áo bên ngoài

Model hiện nay chủ yếu là kiến trúc decoder-only (GPT, Claude, Gemini, Kimi), nhiều model dùng MoE; sau pre-training còn các bước căn chỉnh (SFT, RLHF/DPO) và luyện suy luận (reasoning training, từ ~2025).

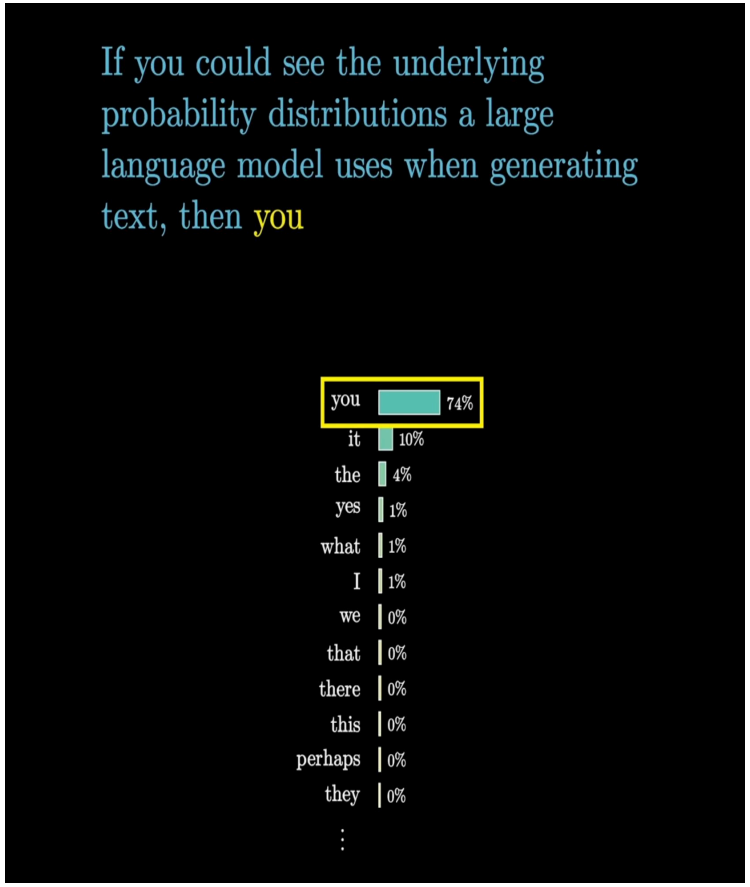
Bên trong Transformer: đầu ra luôn là một phân bố xác suất

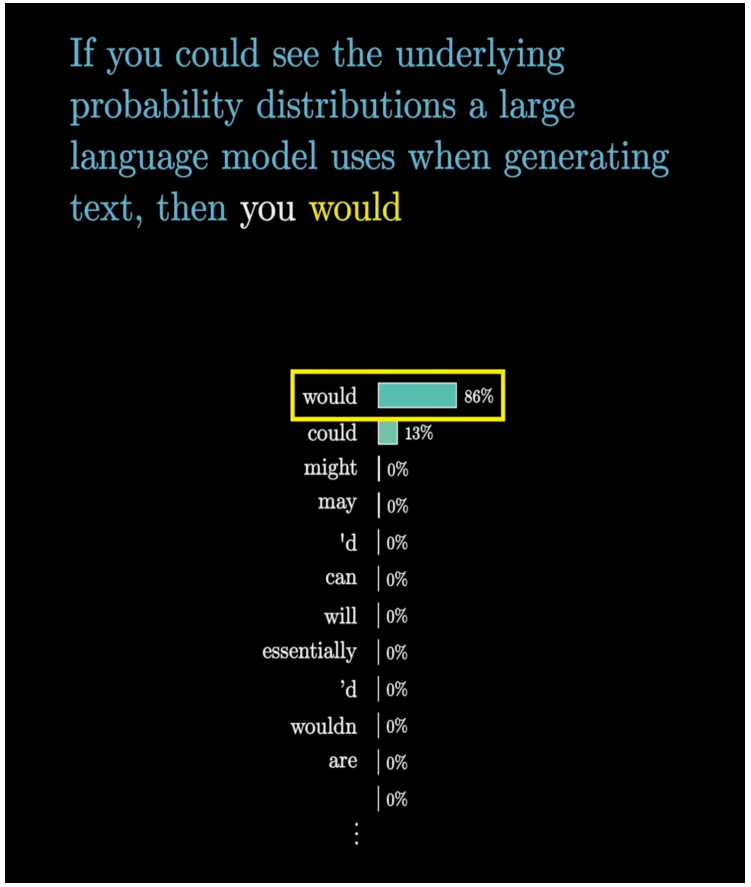


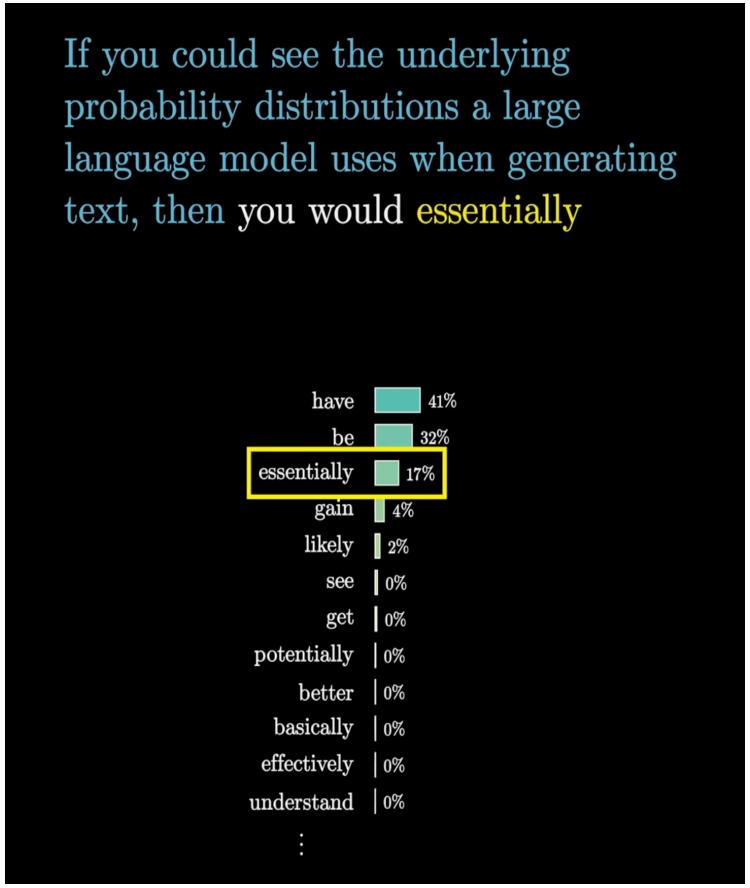
*Với mọi ngữ cảnh, model chấm điểm **MỌI từ** trong từ vựng — "land" 22%, "forest" 9%... — rồi chọn theo xác suất đó*

Transformers, the tech behind LLMs - 3Blue1Brown

Sinh văn bản = đoán → nối vào câu → đoán tiếp







Mỗi token mới được nối vào ngữ cảnh, rồi model chạy lại từ đầu — vòng lặp predict → append → rerun

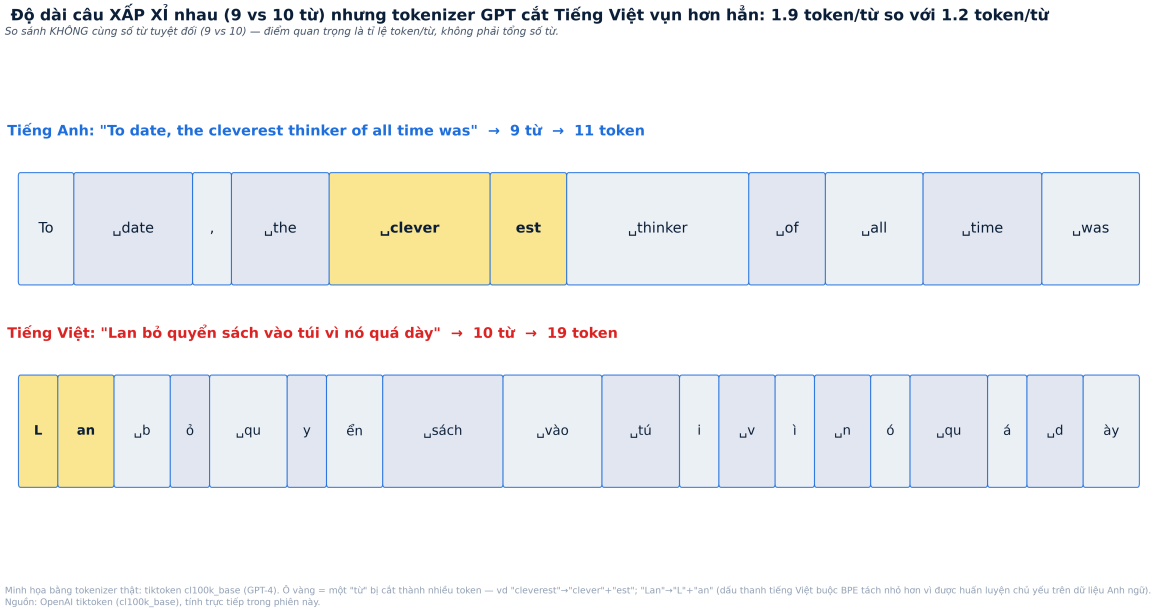
Transformers, the tech behind LLMs - 3Blue1Brown

Token: model không đọc "từ", model đọc mảnh chữ

Model không nhìn từ nguyên vẹn. Nó cắt văn bản thành các **mảnh nhỏ gọi là token**: có từ là một mảnh, có từ vỡ ba bốn mảnh, cả dấu câu và khoảng trắng cũng là mảnh.

Ví dụ: *"Hello world"* ≈ 2 token, nhưng *"Xin chào"* có thể tới 3–4 token.

Tiếng Việt, code, JSON tốn token hơn tiếng Anh thường — vì dấu thanh, ký tự đặc biệt và cấu trúc bị cắt nhỏ ra.



*Mọi thứ model làm đều quy ra token — và **mỗi token đều có giá**. Nhớ điều này khi sang phần chi phí.*

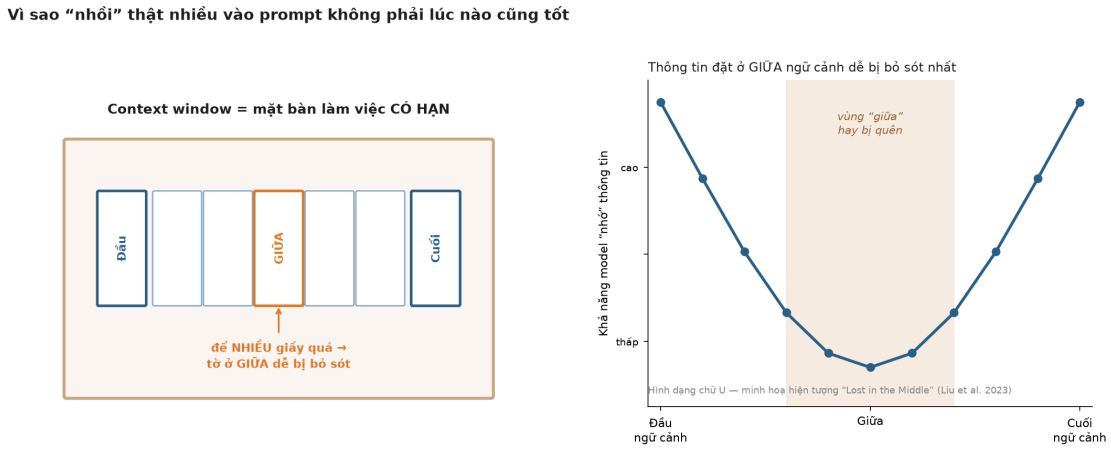
Thử trực tiếp: platform.openai.com/tokenizer · Số token chính xác phụ thuộc tokenizer của từng model.

Context: bàn làm việc có hạn của model

Mỗi lần trả lời, model chỉ nhìn được một lượng chữ có hạn — gọi là **context**. Hãy hình dung một **bàn làm việc**: mọi thứ muốn model "thấy" phải bày lên bàn.

Quy đổi: 128K token ≈ một cuốn sách 300 trang; 1M token ≈ 4–5 cuốn sách trên bàn cùng lúc.

Bàn đầy quá thì đồ ở **giữa bàn dễ bị bỏ sót** — đặt điều quan trọng ở giữa một prompt rất dài, model có thể "quên" mất.



Context càng dài càng tốn tiền và càng chậm — bàn rộng không có nghĩa là dùng tốt

Hiện tượng “quên phần giữa”: Liu et al. (2023), “Lost in the Middle” — arxiv.org/abs/2307.03172. Model thế hệ mới đã cải thiện đáng kể nhưng chưa hết hẳn.

Attention: mỗi từ được “nhìn sang” những từ quan trọng khác

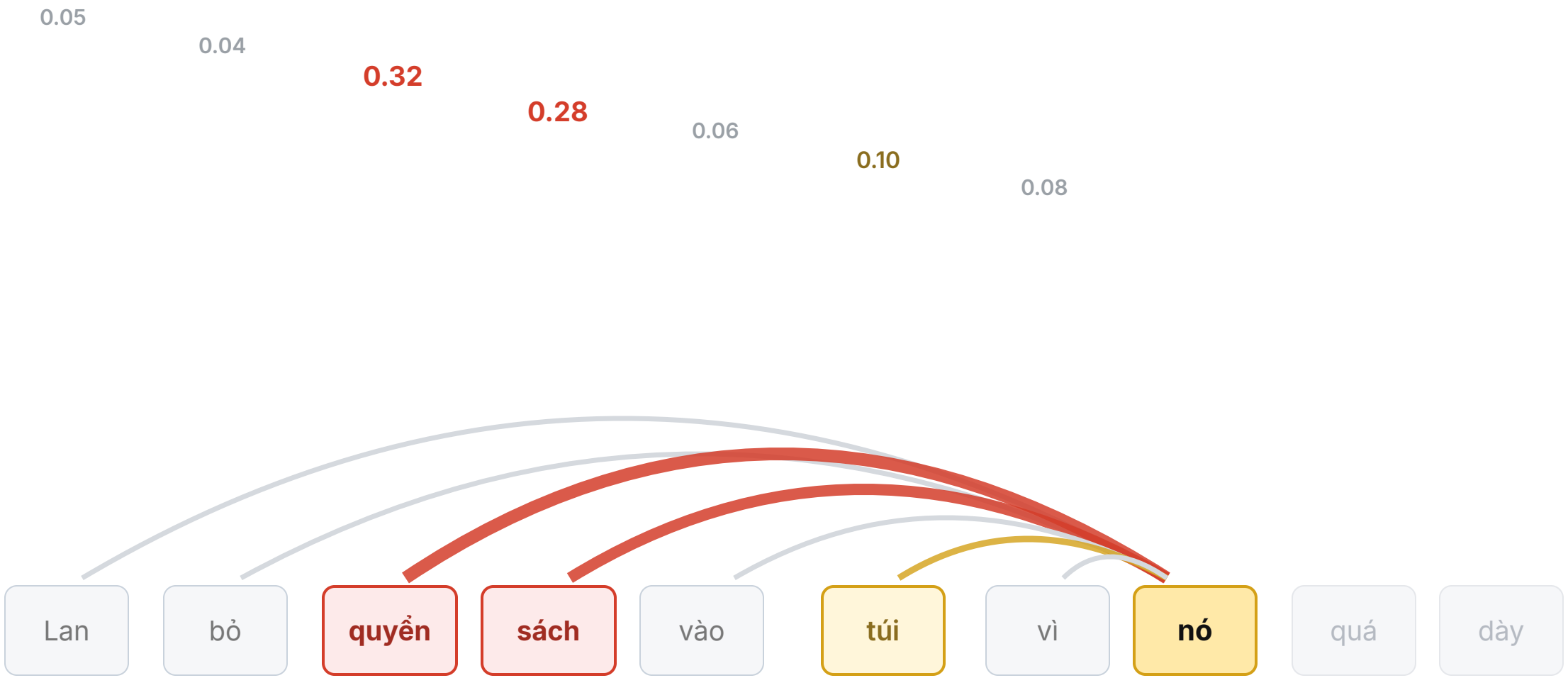
Thay vì đọc tuần tự từng chữ, cơ chế **attention** cho phép mỗi token:

- **Chủ động “quay đầu”** nhìn lại các token trước đó trong câu
- **Chấm điểm mức độ liên quan** của từng token đối với nghĩa của mình
- **Khóa nghĩa theo ngữ cảnh** — “nó” là quyển sách hay cái túi, tùy theo nó chú ý vào từ nào

Đây chính là chữ T trong GPT — và là lý do model hiểu ngữ cảnh tốt hơn hẳn các thế hệ trước

Video minh họa: **Attention in transformers, step-by-step - 3Blue1Brown**

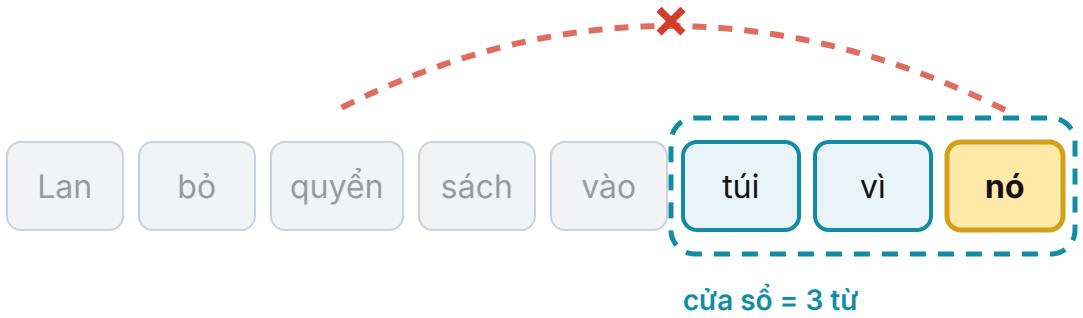
Minh họa khái niệm: token "nó" cần "chú ý" (attention) tới token nào để hiểu đúng nghĩa?



"Lan bỏ quyển sách vào túi vì nó quá dày" — muốn biết "nó" = quyển sách hay cái túi, mô hình so khớp "nó" với TẤT CẢ token trước đó, không chỉ token liền kề. Cung càng dày/đậm = trọng số attention càng cao (ở đây: hướng mạnh về "quyển"+"sách", không phải "túi").

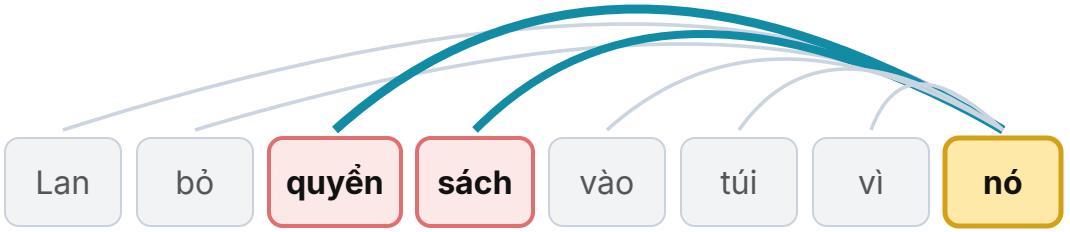
Nhìn lân cận hay nhìn toàn cảnh?

Convolution — cửa sổ nhỏ quanh mỗi từ



"nó" muốn hiểu nghĩa thì phải nhìn tới "quyển/sách" — **nhưng chúng nằm ngoài cửa sổ** → mối liên hệ xa bị cắt.

Attention — mọi từ đều trong tầm nhìn



"nó" nhìn lại **toàn bộ câu** và tự chọn từ quan trọng — nét đậm ở "quyển", "sách" nghĩa là **chú ý mạnh vào đó**.

*Cửa sổ nhỏ thì nhanh nhưng mù xa — **attention đổi tốc độ lấy khả năng giữ ngữ cảnh dài**, và đó là bước ngoặt*

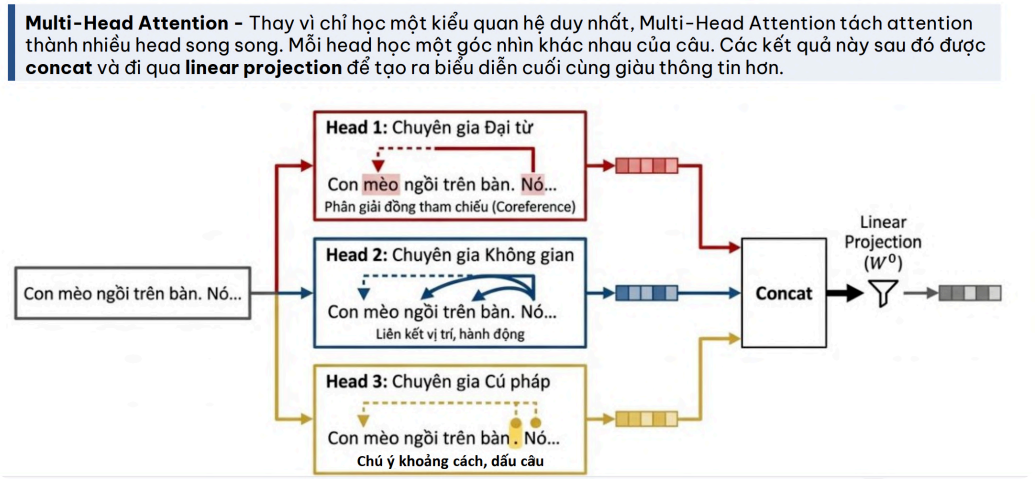
Ẩn dụ so sánh từ bài nói của Łukasz Kaiser (OpenAI, đồng tác giả "Attention Is All You Need")

Multi-head: cùng một câu, nhiều con mắt chuyên môn nhìn song song

Attention không chỉ có một "con mắt". Model có **nhiều con mắt chuyên môn** nhìn cùng một câu một lúc:

- 👁️ **Con mắt đại từ** — lo việc "nó" là con mèo hay cái bàn.
- 👁️ **Con mắt không gian** — lo việc cái gì nằm trên cái gì.
- 👁️ **Con mắt cú pháp** — lo nhịp câu, dấu câu, khoảng cách.

Mỗi con mắt nhìn một khía cạnh, rồi model **tổng hợp lại** thành hiểu biết đầy đủ hơn về câu.



Một con mắt nhìn được một góc — nhiều con mắt cùng nhìn mới thành "hiểu ngữ cảnh"

Multi-head attention — Vaswani et al. (2017) — arxiv.org/abs/1706.03762

Hiểu attention để dùng AI hiệu quả: quản context = quản sự chú ý

Attention có hạn và có "điểm mù". Vì vậy, cách bạn bày context quyết định model chú ý vào đâu:

1 Đặt điều quan trọng đầu – cuối

Đầu và cuối prompt được chú ý nhiều nhất; đồ ở giữa dễ bị bỏ sót — yêu cầu quan trọng đừng chôn giữa.

2 Giữ bàn làm việc sạch

Context rác = attention rác. Khi chat dài, tóm tắt lại thay vì kéo theo mọi thứ; khi vibe code, đưa đúng file liên quan, không dán cả repo.

3 Cho tra sổ thay vì bắt nhớ

Tài liệu dài: lấy đoạn liên quan nhét vào context (RAG) thay vì trông chờ model nhớ hết hoặc nhét cả cuốn.

Agent mạnh không phải vì context khổng lồ — mà vì nó có tools để lấy đúng thứ vào bàn làm việc đúng lúc

Tham số (parameter): những "khớp nối" model học được

Sau khi luyện xong, những gì model "biết" nằm trong các con số cố định bên trong gọi là **tham số** — hãy hình dung như **khớp nối thần kinh**: luyện càng kỹ, các khớp nối càng được siết đúng.

Tham số không phải thứ bạn chỉnh khi dùng model — nó được đóng gói sẵn trong "bộ não" (file weights). Bạn chỉ chỉnh được context và các núm vặn lúc gọi (như temperature).

2020 — GPT-3

175 tỷ

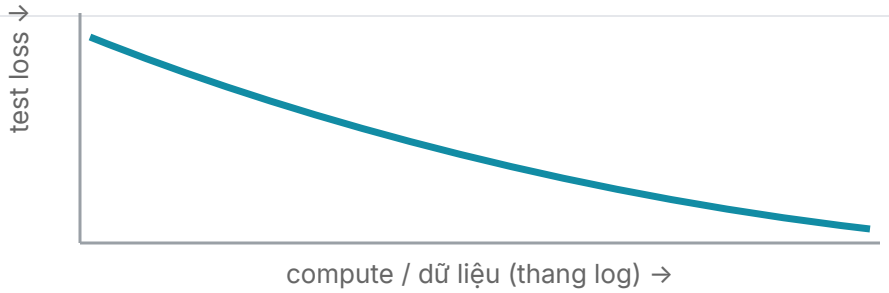
một "bác sĩ đa năng" — mọi token đều đi qua toàn bộ khớp nối (dense)

2026 — Kimi K3

2.800 tỷ

một "bệnh viện đa khoa" — mỗi token chỉ gọi vài chuyên gia (MoE)

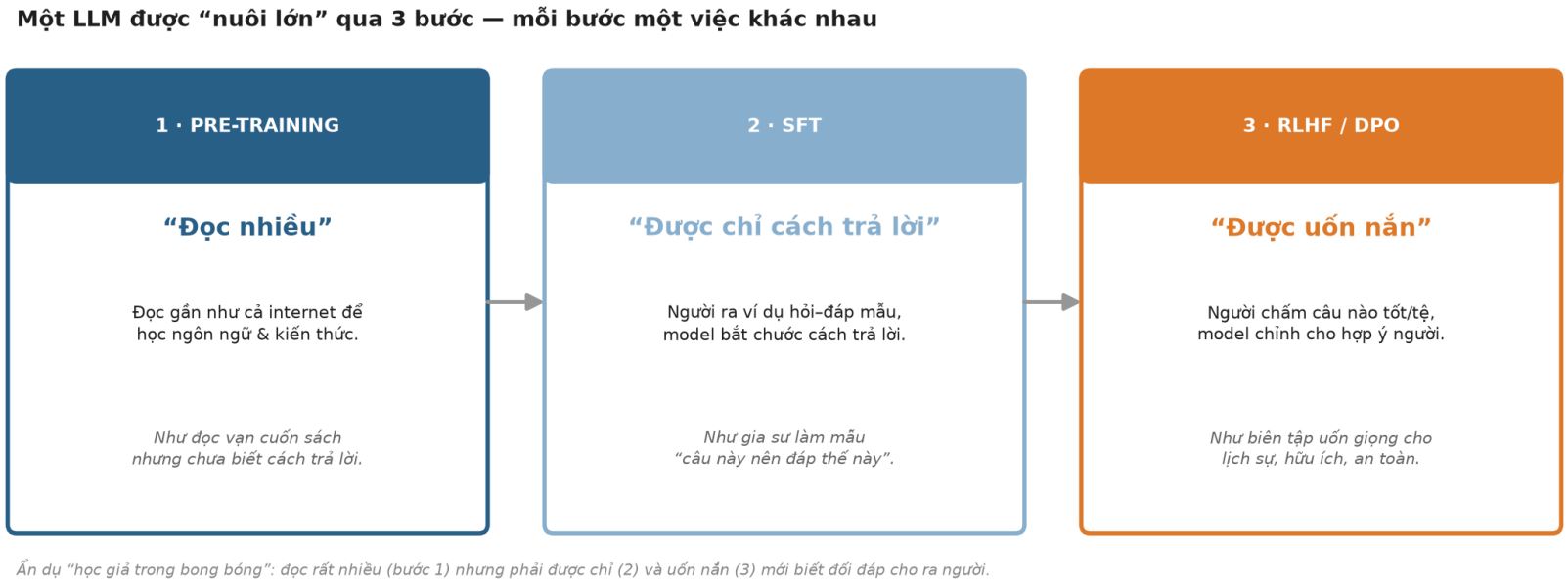
Nhiều tham số ≠ tốn hơn tuyến tính — nhờ MoE, bệnh viện lớn gấp 16 lần mà chi phí mỗi ca khám gần như không đổi



Luật chơi 2020–2024: cứ thêm compute + dữ liệu là model khôn lên **một cách dự đoán được** (scaling law, Kaplan et al. 2020)

MoE: Shazeer et al. (2017) — arxiv.org/abs/1701.06538 · Kimi K3 (16/7/2026): ~2.8 nghìn tỷ tham số MoE — k3-kimi.com

LLM được tạo ra như thế nào? — đọc nhiều, được chỉ, được uốn nắn, luyện đề

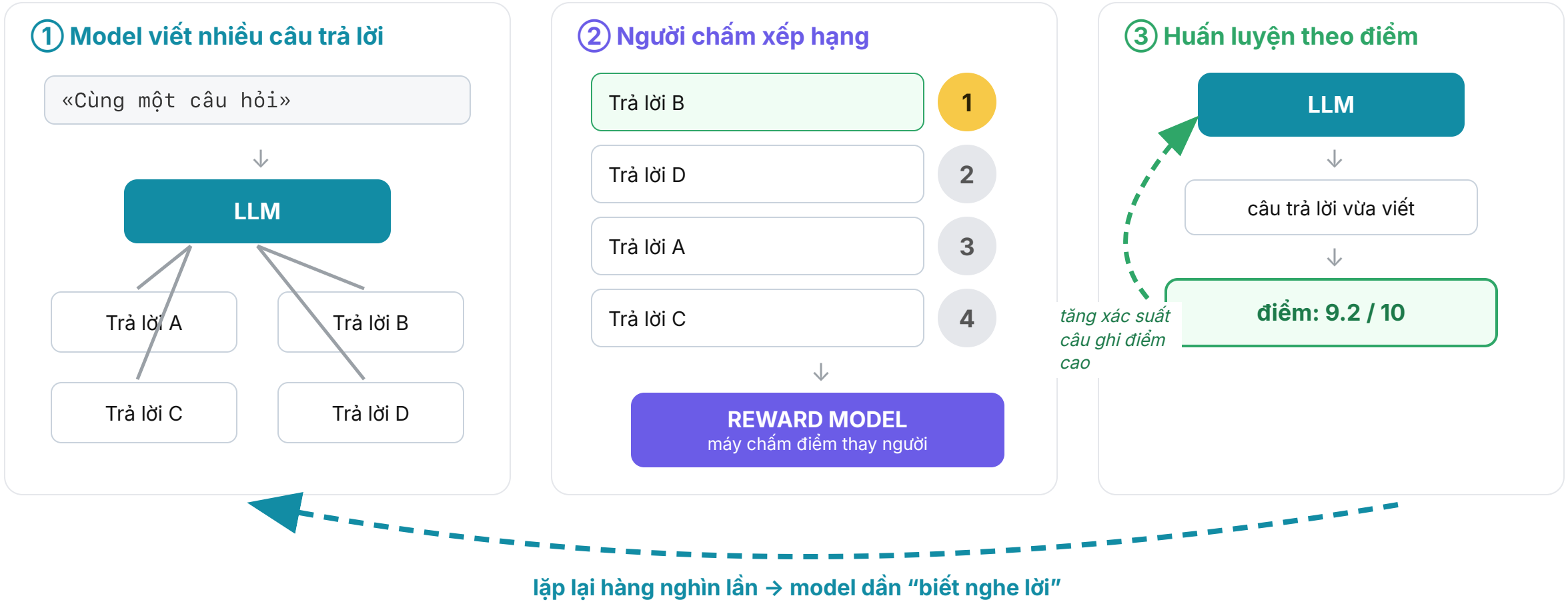


① **Pre-training — "đọc cả thư viện":** học tiếng nói và kiến thức từ hàng nghìn tỷ token. ② **SFT — "được chỉ cách trả lời":** học theo ví dụ mẫu để ra dáng trợ lý. ③ **RLHF/DPO — "được uốn nắn":** học theo phản hồi con người, an toàn và dễ chịu hơn. ④ **Luyện suy luận — "giải đề tự chấm" (từ ~2025):** luyện toán/code có đáp án kiểm chứng được → model biết làm nháp trước khi trả lời.

Đọc vạn cuốn sách chưa chắc biết trả lời phỏng vấn — đó là lý do cần bước ②, ③, ④

Ouyang et al. (2022), InstructGPT — arxiv.org/abs/2203.02155 · Rafailov et al. (2023), DPO — arxiv.org/abs/2305.18290 · RLVR: RL with verifiable rewards.

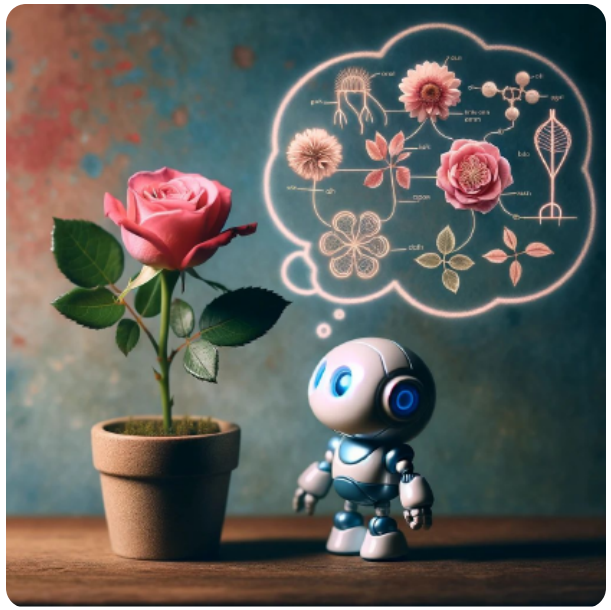
RLHF: ba bước uốn cỗ máy đoán token thành trợ lý biết nghe lời



Cỗ máy đoán token + điểm xếp hạng của con người → trợ lý helpful · harmless · honest

Ouyang et al. (2022), "Training language models to follow instructions with human feedback" (InstructGPT) — arxiv.org/abs/2203.02155 · DPO (cách đơn giản hơn, 2023) — arxiv.org/abs/2305.18290

LLM có thực sự “hiểu” — hay chỉ là vẹt thống kê?



?

1 Mô hình thế giới bên trong?
nén thế giới thành biểu diễn có cấu trúc

2 Nôn lại dữ liệu huấn luyện?
chỉ ghép các mẫu chữ theo xác suất

*Chỉ đoán token tiếp theo thôi — vậy sao trông giống đang **hiểu** mình nói gì?*

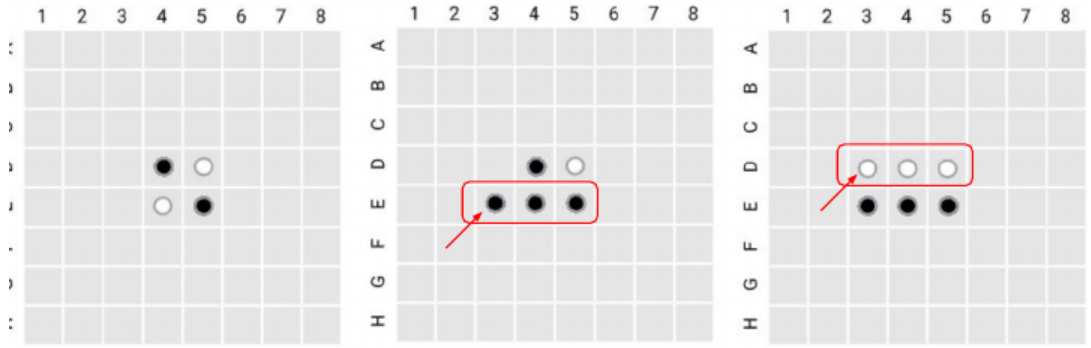
Tranh luận từ Turing (1950), “Computing Machinery and Intelligence”, Mind · “Stochastic parrots”: Bender, Gebru, McMillan-Major & Shmitchell (2021), FAccT’21 · Hình minh họa: Martin Wattenberg (Harvard)

Thí nghiệm Othello-GPT: dạy cỗ máy đoán chữ chơi cờ

Đầu vào duy nhất: chuỗi token biên bản ván cờ

```
C4 C3 D3 C5 D6 F4 B4 C6 B5 B3 B6 E3 C2
A4 A5 A6 D2 ?
```

- ✕ **Không** được dạy luật chơi
- ✕ **Không** hề thấy bàn cờ 8×8
- ✕ **Không** biết quân trắng–đen — chỉ thấy chuỗi ký tự



Ba trạng thái bàn cờ thật mà con người nhìn được — còn model thì không bao giờ thấy.

*Câu hỏi: **đoán được nước đi tiếp theo không?** — chỉ từ chuỗi ký tự đó thôi*

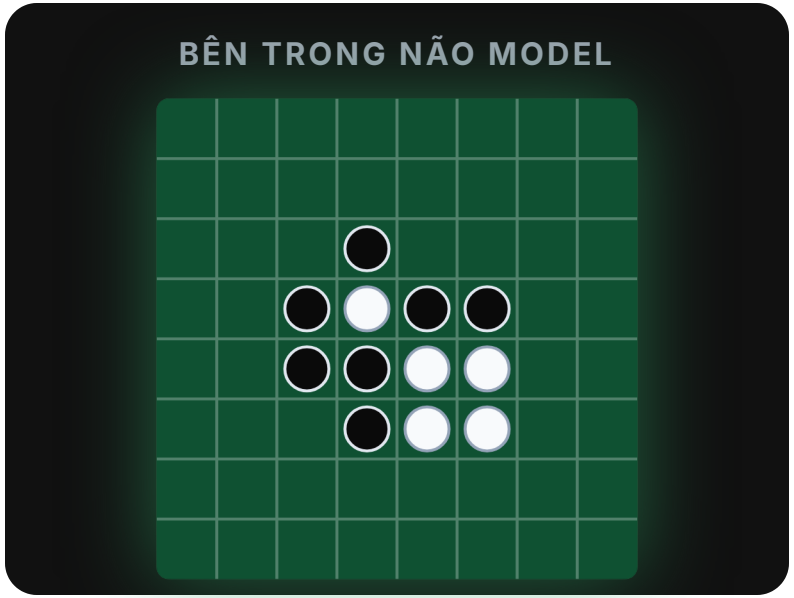
Li, Hopkins, Bau, Viégas, Pfister & Wattenberg (2023), "Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task", ICLR 2023 (Oral) · arXiv:2210.13382

Muốn đi hợp lệ, nó buộc phải tự dựng lại bàn cờ trong đầu

ĐẦU VÀO DUY NHẤT

```
C4 C3 D3 C5 D6 F4 B4 C6 B5
B3 B6 E3 C2 A4 A5 A6 D2 ?
```

chuỗi token biên bản ván cờ — **không luật chơi, không bàn cờ, không quân trắng–đen**



một "bàn cờ ẩn" tự hình thành — không ai dạy

ĐẦU RA

F5

✓ **nước đi hợp lệ**

tỷ lệ đi sai luật chỉ **~0.01%**

đi hợp lệ ⇒ phải **biết** ô nào trắng, ô nào đen, ô nào trống

*Không ai cho nó xem bàn cờ — để đoán đúng token tiếp theo, cỗ máy **tự xây một mô hình thế giới bên trong***

Mở hộp đen kiểm chứng: bàn cờ có thật trong não model

Nhóm nghiên cứu gắn 64 "que thử" (probe) vào bên trong model — mỗi que hỏi một ô: **"ô này đang trắng, đen, hay trống?"**

1 Que thử đọc được toàn bộ bàn cờ

Từ activation bên trong, probe đọc ra trạng thái từng ô — chính xác vượt xa mức ngẫu nhiên, và càng giữa ván càng chính xác.

2 Lật một quân trong "đầu" nó → nước đi đổi theo

Khi can thiệp lật màu một quân cờ trong biểu diễn bên trong, các nước đi hợp lệ model dự đoán **đổi theo đúng luật** — tức nó thật sự **dùng** bàn cờ đó để chơi.

Model chỉ đoán token tiếp theo — nhưng để đoán giỏi, nó tự xây một mô hình thế giới bên trong

Li et al. (2023), ICLR 2023 (Oral) — arXiv:2210.13382 · Bản đọc dễ hơn: thegradient.pub/othello

Giới hạn bẩm sinh: học giả trong bong bóng

Bong bóng thời gian

Model bị "đóng băng" tại ngày ngừng đọc. Chuyện sau đó nó không biết — trừ khi bạn cung cấp thêm (knowledge cutoff).

Nói chắc như đúng rồi

Model tối ưu cho câu **nghe hợp lý**, không phải tra sự thật — nên có thể tự tin mà sai (hallucination).

Bàn làm việc có hạn

Context có trần; quá dài vừa tốn tiền vừa dễ bỏ sót thông tin ở giữa.

> *"Why does it work? We don't know — a lot here are intuitions, not theorems or truths."* — Łukasz Kaiser, đồng tác giả "Attention Is All You Need" (OpenAI)

*Đây không phải lỗi tạm thời — đó là **bản chất của cỗ máy đoán token**. Vì vậy ta cần prompt tốt, context sạch, tra sổ (RAG), tools, và luôn kiểm chứng.*

“Biết nhiều” khác “làm được”: dữ liệu mới và hành động thật cần tools/retrieval/workflow — nền của các ngày sau.

Vì sao model vẫn sai: nó rất giỏi học vẹt đường tắt

1 Phân loại spam

Model thực chất đã học:

"đếm số hyperlink trong email"

Email sạch nhưng nhiều link → vẫn bị gán spam

2 Câu chủ quan vs khách quan

Model thực chất đã học:

"có phải câu trích từ film review không"

Ăn gian bằng nguồn gốc câu, không phải nội dung câu

3 Suy luận ngôn ngữ (MNLI)

Model thực chất đã học:

"câu có động từ phủ định"

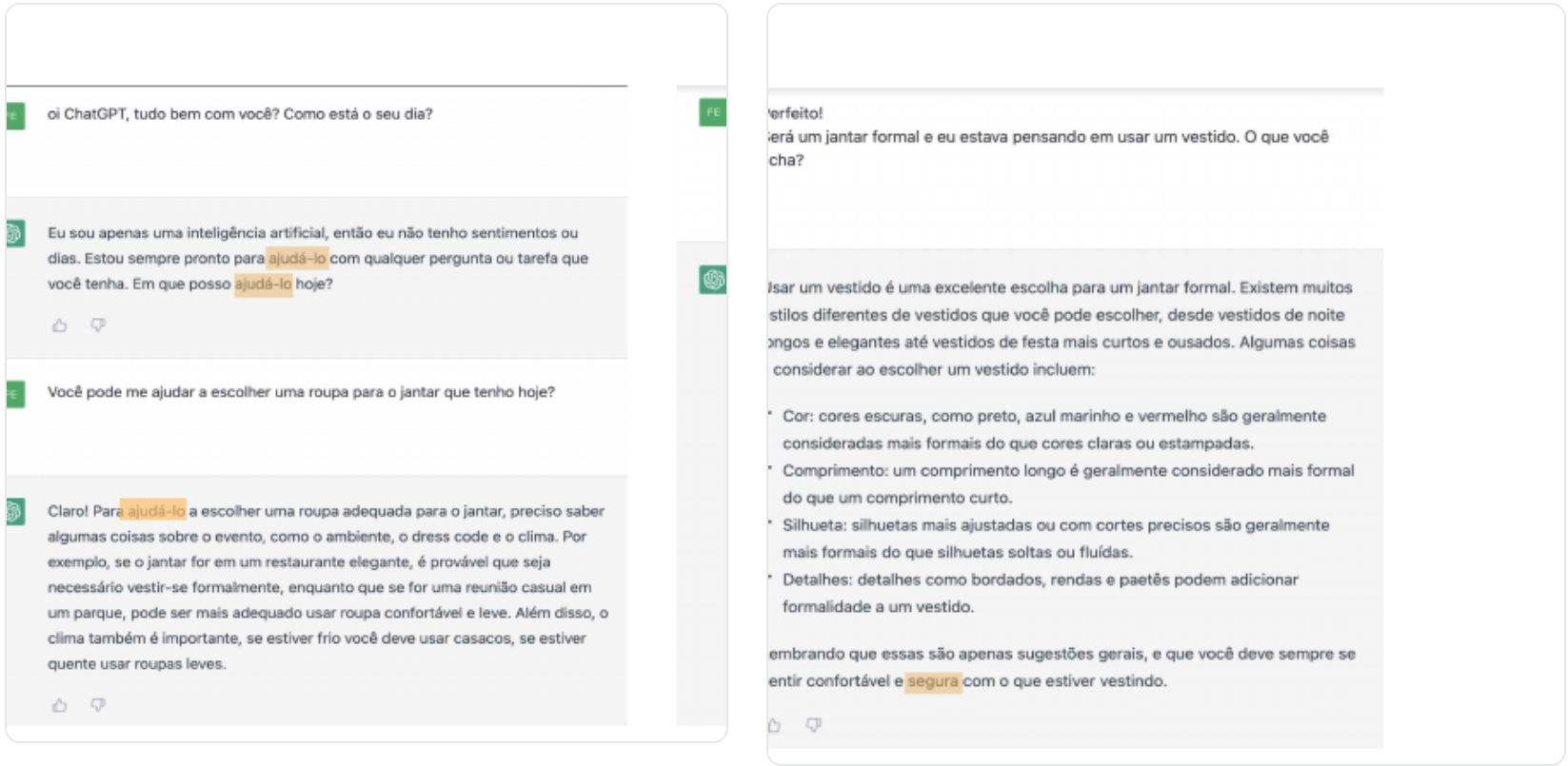
Đổi cấu trúc dữ liệu test là điểm tụt ngay

Ba "đường tắt" (spurious cues) trên do **chính LLM tự động phát hiện** và mô tả bằng ngôn ngữ tự nhiên — trên quy mô 675 bài toán thật của benchmark OpenD5.

Benchmark cao ≠ model hiểu đúng thứ bạn tưởng — luôn test trên dữ liệu của chính mình

Zhong, Snell, Klein & Steinhardt (2022), "Describing Differences between Text Distributions with Natural Language", ICML 2022 · Zhong et al. (2023), "Goal Driven Discovery of Distributional Differences via Language Descriptions" (OpenD5), NeurIPS 2023

Model không chỉ mô hình hóa thế giới — nó mô hình hóa cả BẠN



ChatGPT nói tiếng Bồ Đào Nha với Fernanda Viégas: đầu hội thoại nó dùng động từ giống đực ("ajudá-**lo**").

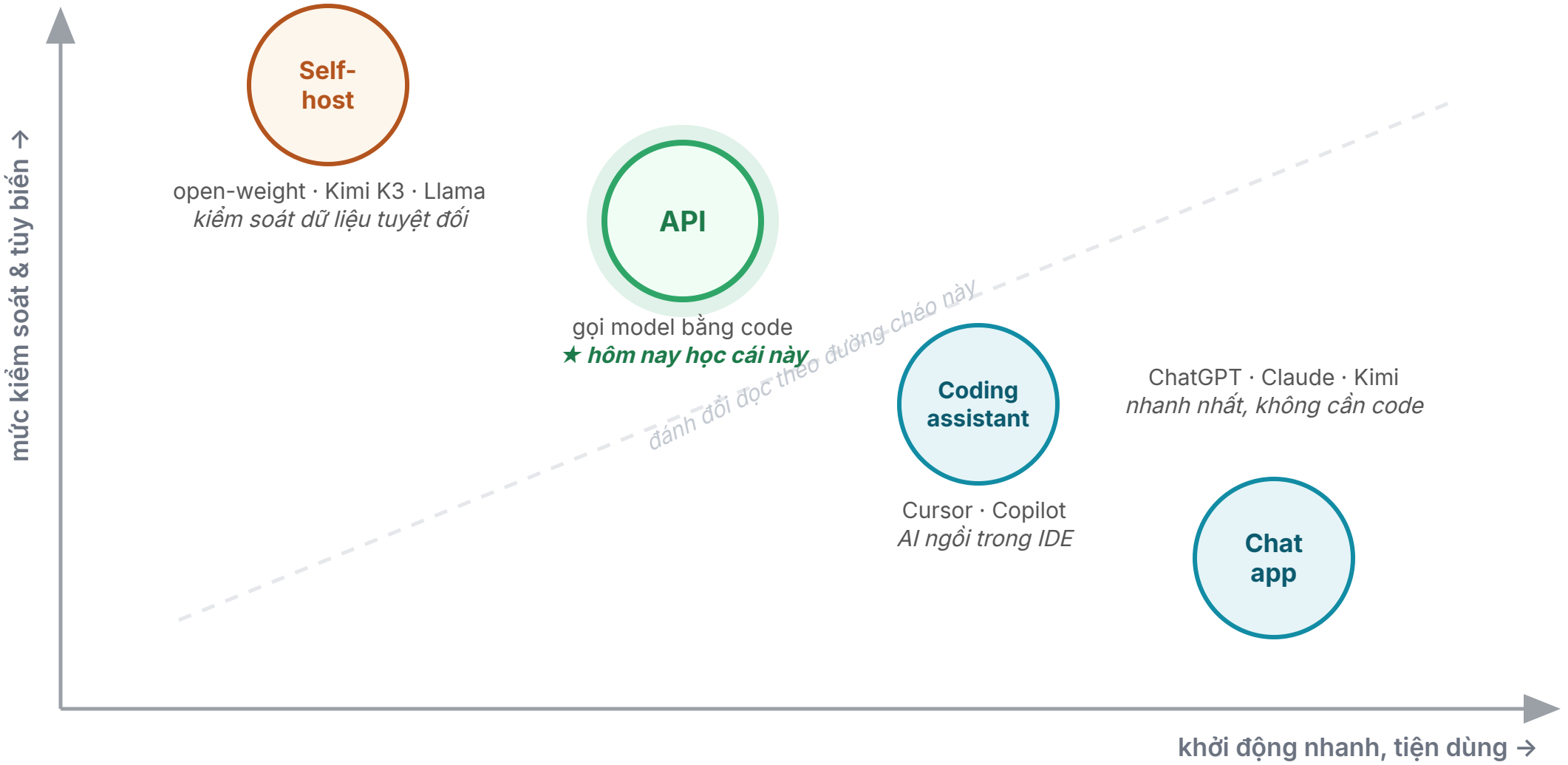
Ngay khi bà nhắc đến chiếc **váy** (vestido), câu sau model chuyển sang tính từ giống cái ("segu**ra**") — nó đã ngầm đoán giới tính người dùng.

Không ai bảo nó làm vậy. Từ cách bạn viết, model tự dựng một "hồ sơ" về bạn — và hồ sơ đó ảnh hưởng câu trả lời.

Cách bạn viết prompt cũng đang nói cho model biết bạn là ai — đó là lý do persona và ngữ cảnh trong prompt rất đáng giá

Quan sát của Fernanda Viégas, kể trong bài nói “Models Within Models” — Martin Wattenberg (Harvard) · Liên hệ: Andreas (2022), “Language Models as Agent Models” — arxiv.org/abs/2212.01681

Bốn cách chạm vào LLM: tiện bao nhiêu, kiểm soát bấy nhiêu



Cùng một bộ não nền, bốn mức quyền truy cập — mức truy cập quyết định bạn tùy biến được tới đâu

Nghịch để tin: tự tay bóc GPT-2 trong trình duyệt

Mở **Transformer Explainer** (nhóm 2–3 bạn một máy, đã tải sẵn), rồi làm 3 việc:

① Gõ một câu, xem nó bị cắt thành token thế nào.
② Vặn **temperature** từ 0 lên cao, chạy lại vài lần, nhìn bảng xác suất đổi ra sao.
③ Mở **attention map**, bấm vào một token, xem nó "đang nhìn" những token nào.

Hai câu chốt để mang về:
"Temperature đổi cách model CHỌN CHỮ — chứ không đổi kiến thức model có" · "Attention map cho thấy model NHÌN VÀO ĐÂU — chứ không chứng minh model hiểu"

poloclub.github.io/transformer-explainer · Paper: arxiv.org/abs/2408.01919 · GPT-2 small là model minh họa kiến trúc, không phải model mới nhất. Attention map cho thấy tương quan, không chứng minh nhân quả.

PHẦN 04

Từ LLM đến AI Agent

đặt bộ não vào vòng làm việc có mục tiêu và hành động

Chain-of-Thought: chỉ thêm "giấy nháp", từ sai thành đúng

Bài toán: "Có 5 quả bóng tennis. Mua thêm 2 hộp, mỗi hộp 3 quả. Hỏi tổng cộng có bao nhiêu quả?"

Không có nháp — trả lời ngay

Model đọc câu hỏi → bật ra đáp án ngay:

```
"Đáp án là 27 quả."
```

× SAI

Có giấy nháp — "hãy nghĩ từng bước"

```
"Bắt đầu có 5 quả.
Mỗi hộp 3 quả × 2 hộp = 6 quả.
5 + 6 = 11.
Đáp án là 11 quả."
```

✓ ĐÚNG

Cùng một model, cùng một câu hỏi — cho nó được viết nháp từng bước, bản chất suy luận lộ ra

Wei et al. (2022), "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" — arxiv.org/abs/2201.11903 · Đây là mầm của các reasoning model (o1, R1...) và của test-time compute ở các slide sau.

LLM đứng một mình chưa làm được gì nhiều

Prompt tĩnh — một lượt hỏi đáp

Prompt

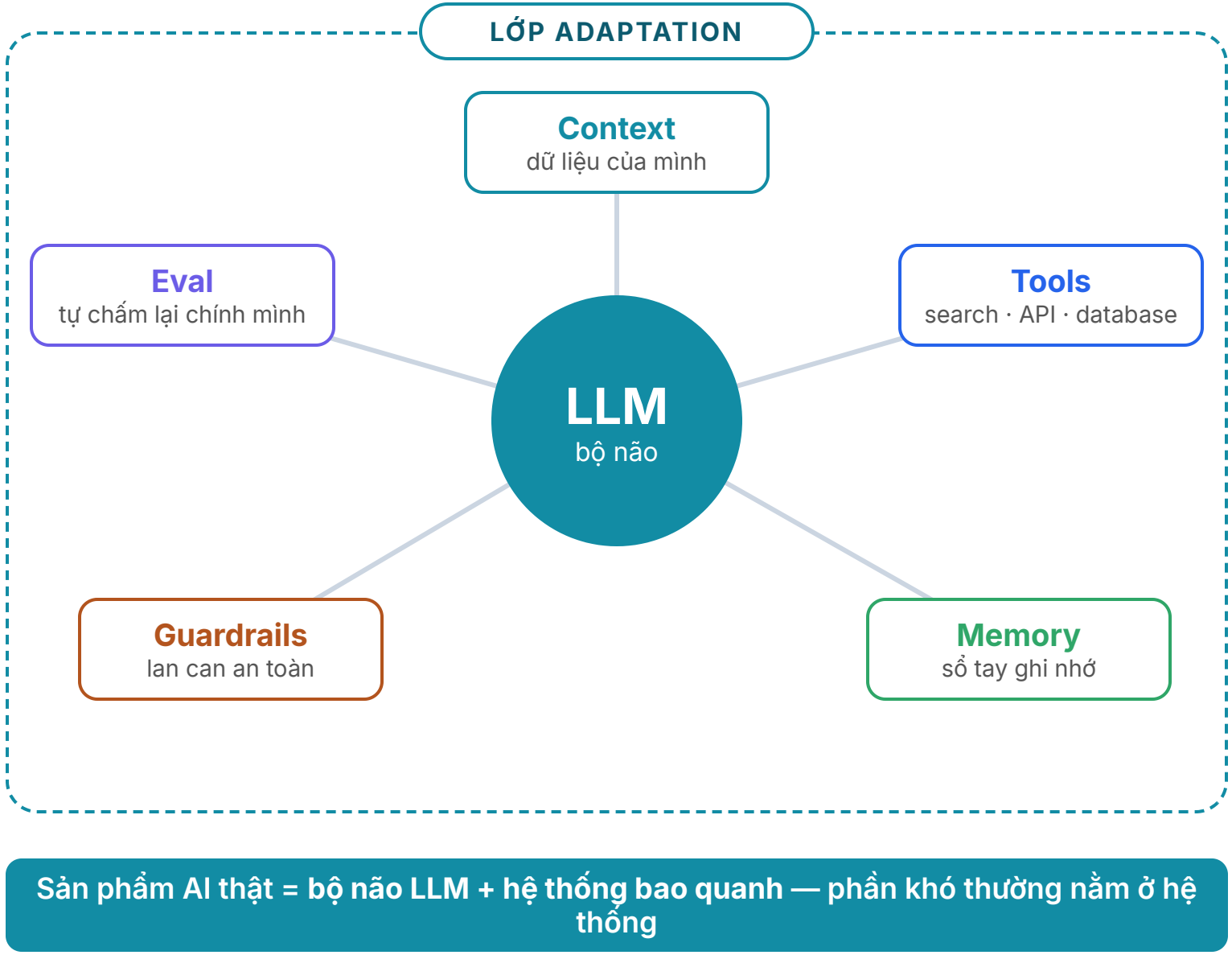
↓

LLM

↓

Response

- × Không dữ liệu mới
- × Không hành động ngoài đời
- × Không nhớ gì sau câu trả lời



Sản phẩm AI thật = bộ não LLM + hệ thống bao quanh — phần khó thường nằm ở hệ thống

Từ LLM đến agent: bốn mức độ — mỗi bậc thêm một năng lực

mức tự chủ & tác động thật tăng dần →

LEVEL 0

Bộ não suy luận

LLM trần — không công cụ, không dữ liệu mới

LEVEL 1

Có kết nối

+ tools: search web, đọc database, gọi API — vượt khỏi bong bóng thời gian

LEVEL 2

Biết lập kế hoạch

+ tự chia mục tiêu thành nhiều bước, dùng nhiều tool nối tiếp, tự kiểm tra kết quả từng bước

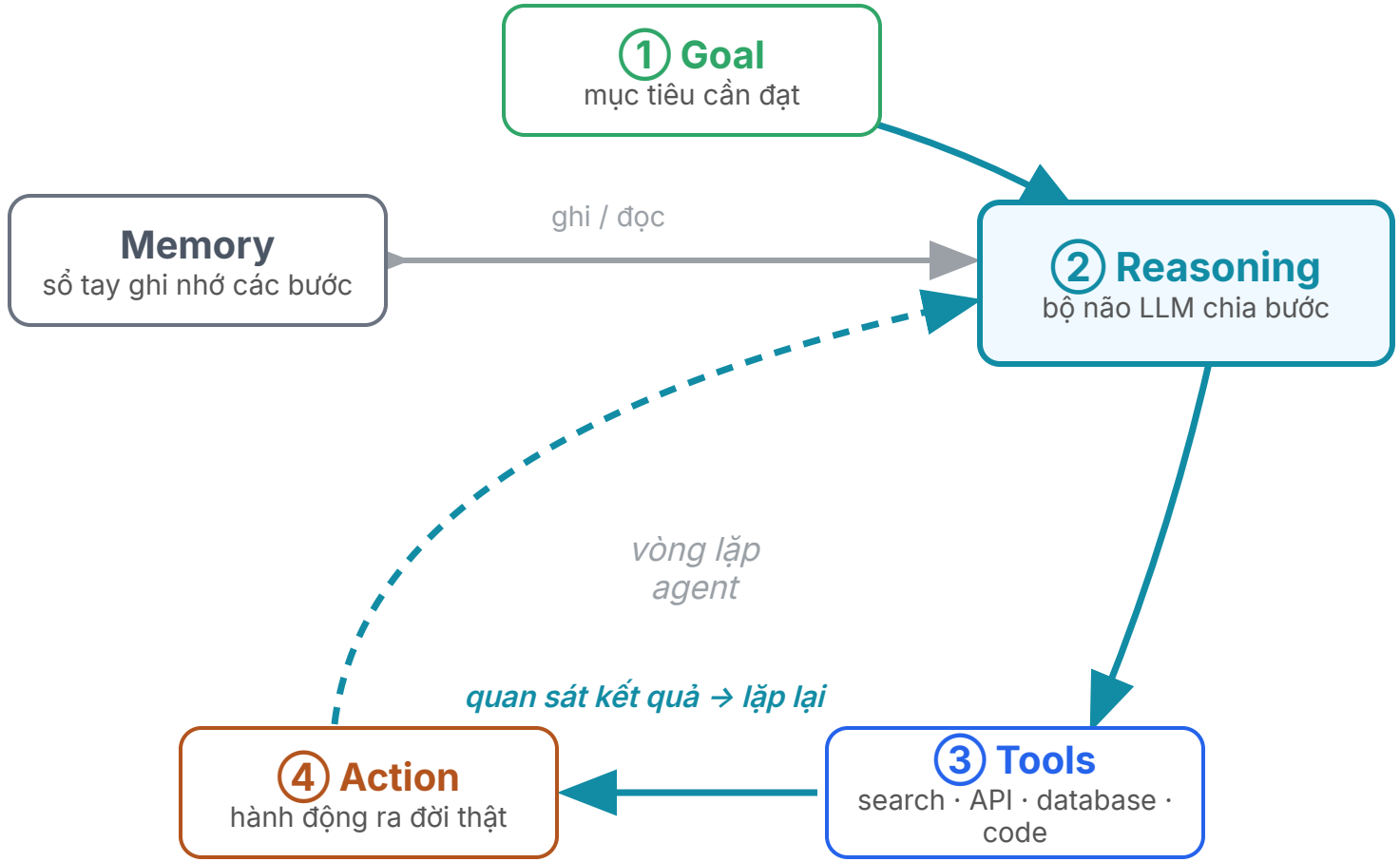
LEVEL 3

Đội agent phối hợp

+ nhiều agent chuyên biệt chia việc như một đội ngũ (multi-agent)

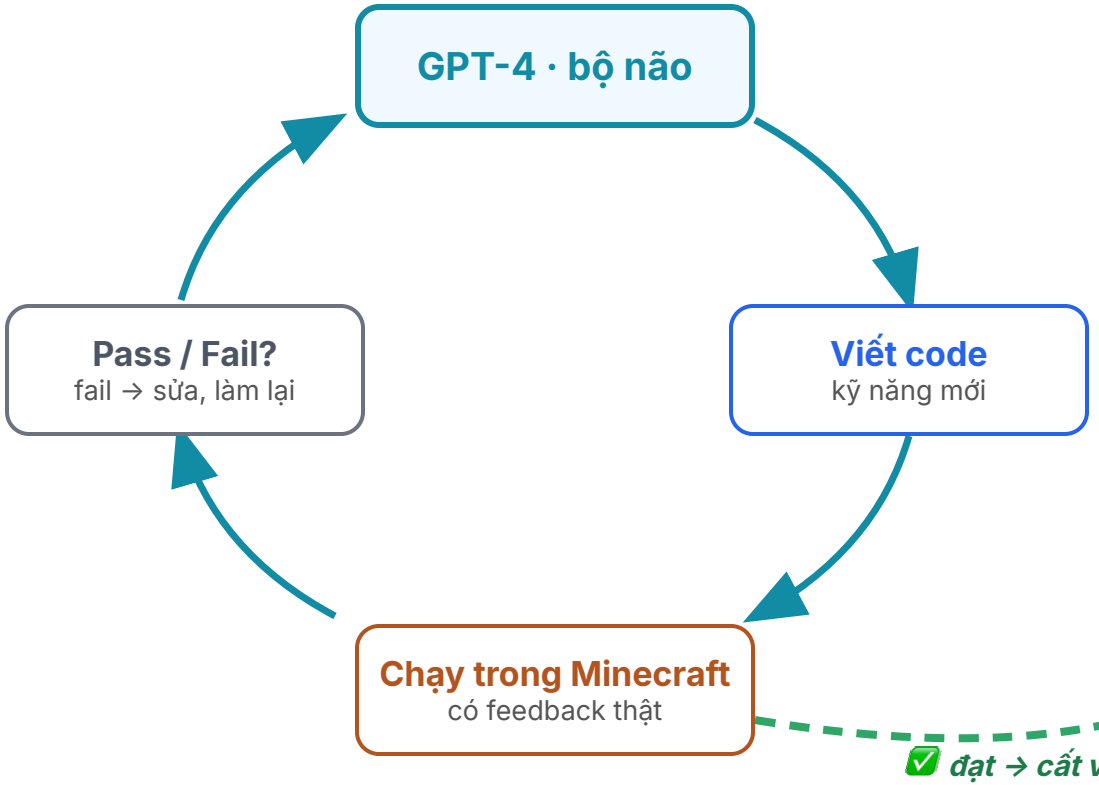
*Agent không phải "một loại model khác" — **đó là LLM được đặt vào vòng làm việc có mục tiêu và hành động***

Giải phẫu một agent: 5 bộ phận là một vòng lặp



Agent = Goal + Reasoning + Tools + Memory + Action — chạy thành vòng lặp cho tới khi xong việc

Voyager: agent tự xây thư viện kỹ năng, rồi sống bằng tái dùng



📚 **THƯ VIỆN KỸ NĂNG — lớn dần theo thời gian**

- `Mine Wood Log`
- `Craft Sword`
- `Craft Furnace`
- `Build Shelter`
- `Hunt Cow`
- `… tự thêm liên tục`

Task mới: «chế tạo bàn chế tác» → truy xuất **top-5 skill liên quan** → làm nhanh hơn, ít sai hơn

Agent giỏi không chỉ vì bộ não to — vì nó tích lũy kỹ năng thành thư viện và tái sử dụng

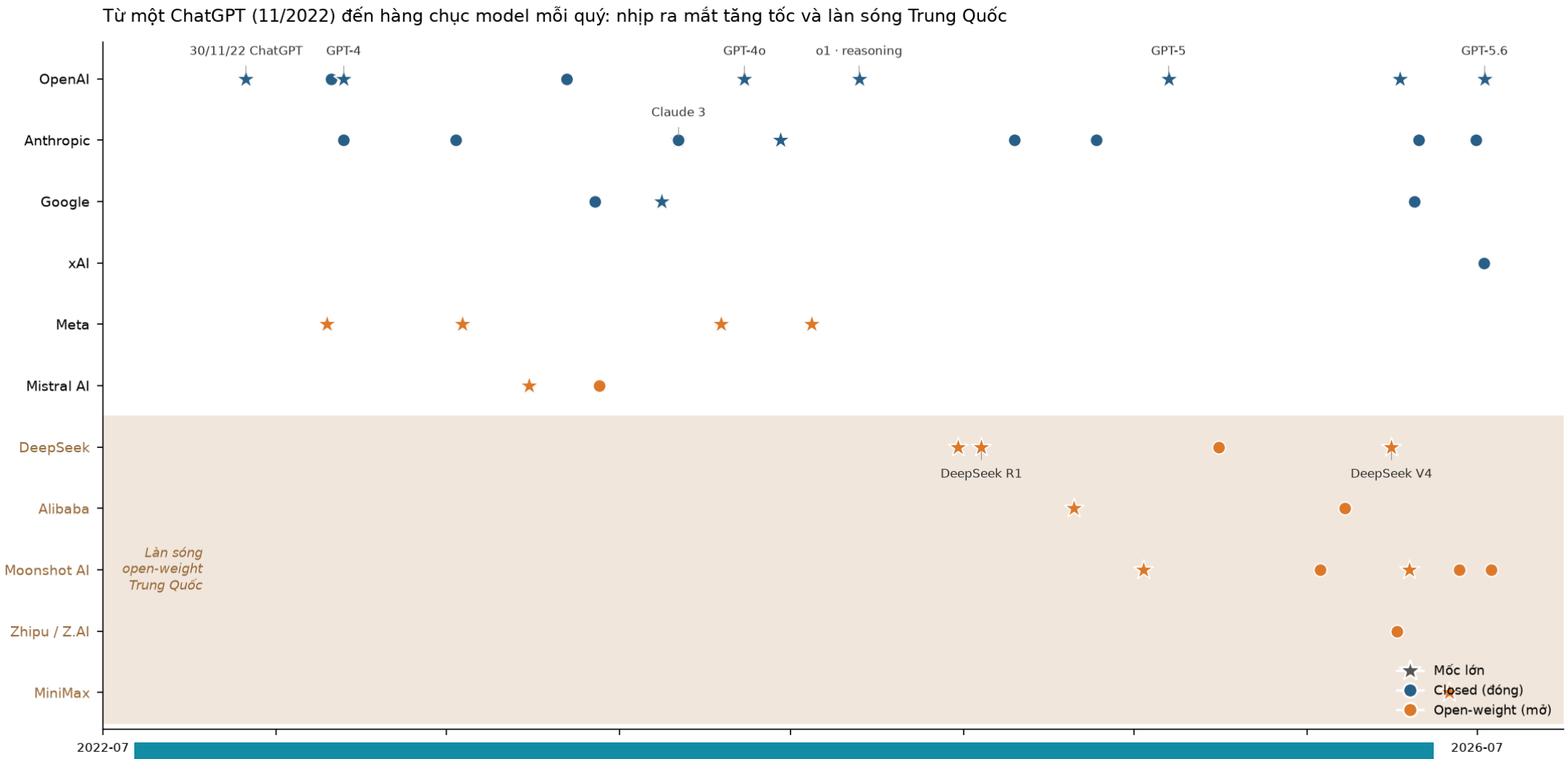
Wang et al. (2023), "Voyager: An Open-Ended Embodied Agent with Large Language Models" — arxiv.org/abs/2305.16291

PHẦN 05

Landscape: model hôm nay

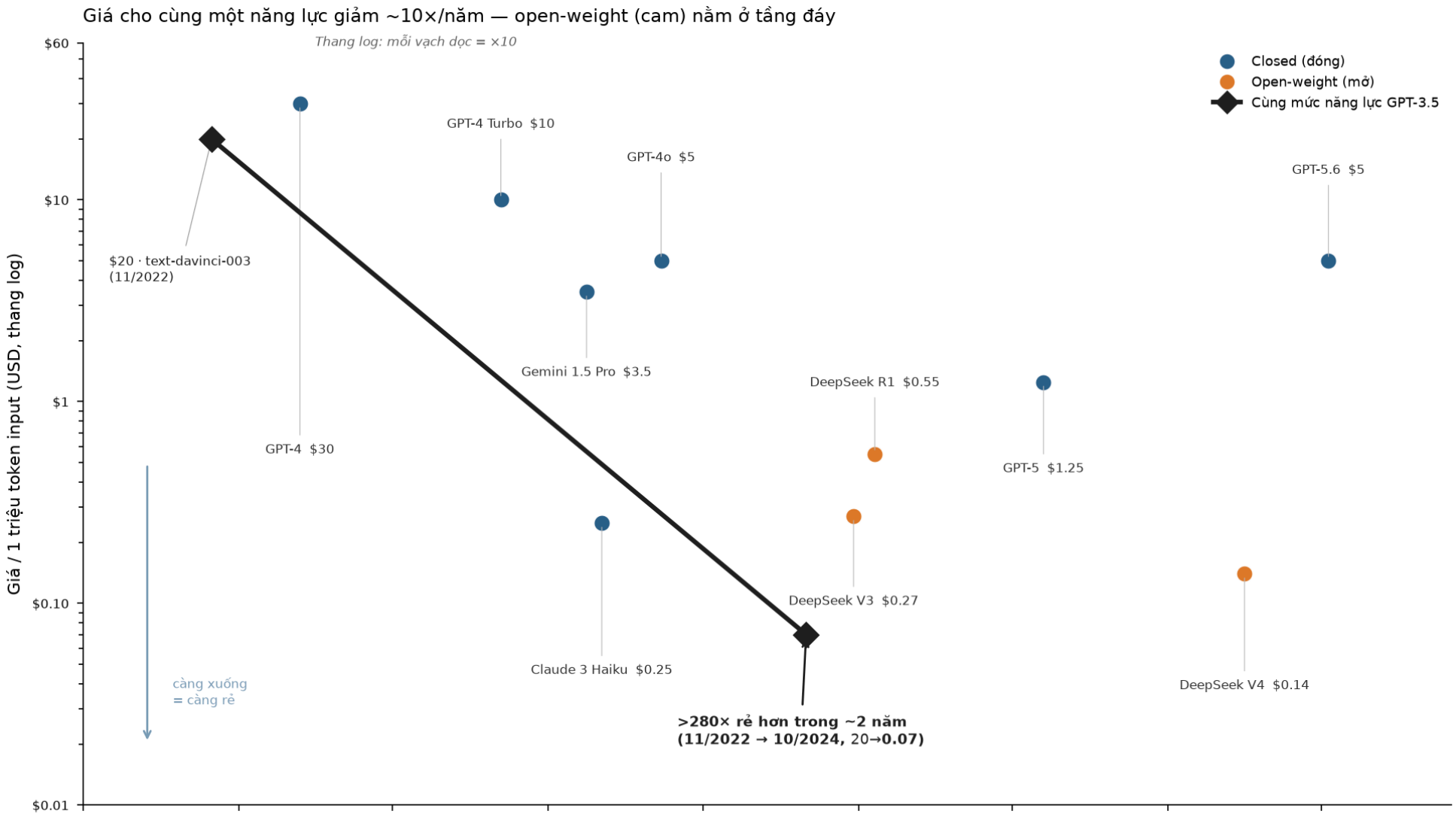
giá rơi, năng lực hội tụ, và cuộc đua đang diễn ra

2022 đến nay: tốc độ ra model tăng chóng mặt



Mỗi năm có hàng chục model đáng chú ý — đừng học thuộc tên, hãy học quỹ đạo

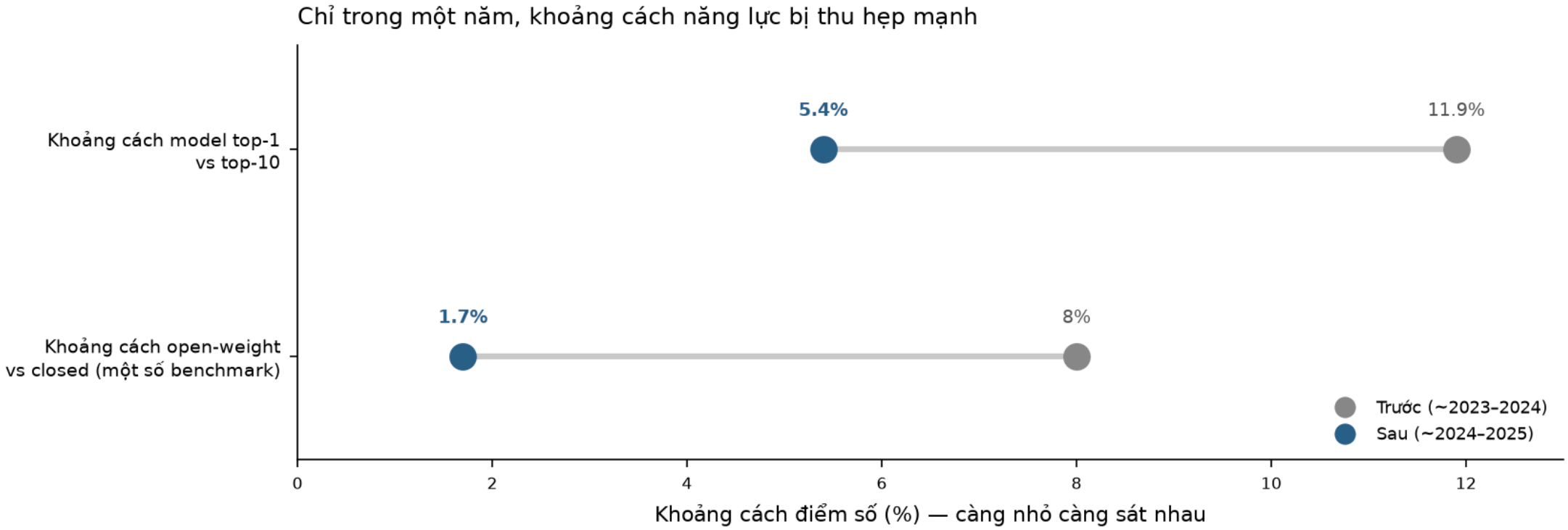
Cùng một mức năng lực, giá rơi khoảng 10 lần mỗi năm



Việc năm ngoái phải dùng model đắt nhất — năm nay model rẻ đã làm được

Tổng hợp từ bảng giá các nhà cung cấp, 2023–2026.

Năng lực hội tụ — và model mở đang bắt kịp model đóng

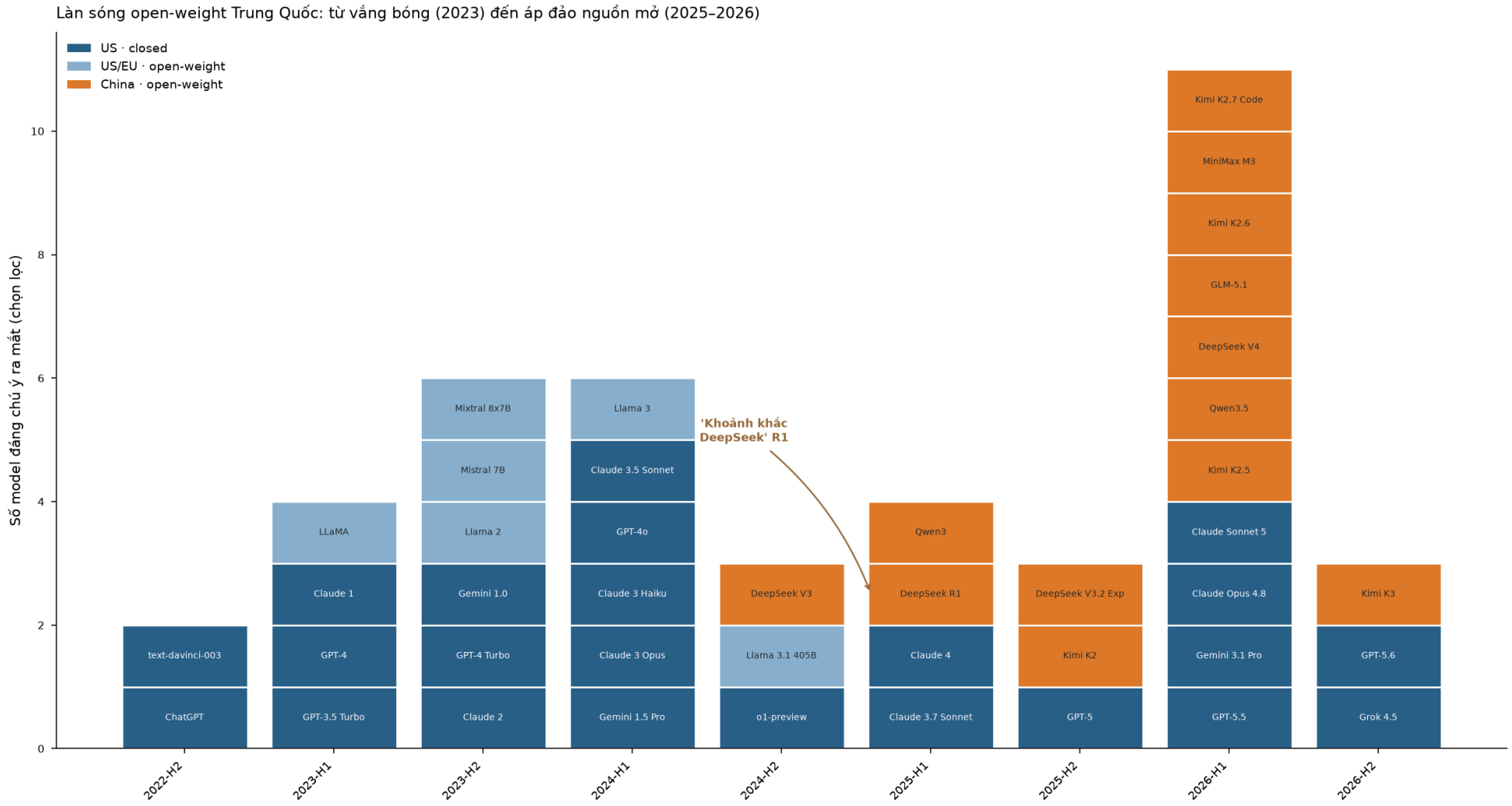


Nguồn: Stanford HAI AI Index 2025

Không còn một model bỏ xa phần còn lại — chọn model là bài toán phương pháp, không phải bài toán nhớ tên

Sắc thái mới: AI Index ghi nhận frontier hội tụ nhưng khoảng cách mở-đóng hơi nới lại 2025 — hai.stanford.edu/ai-index

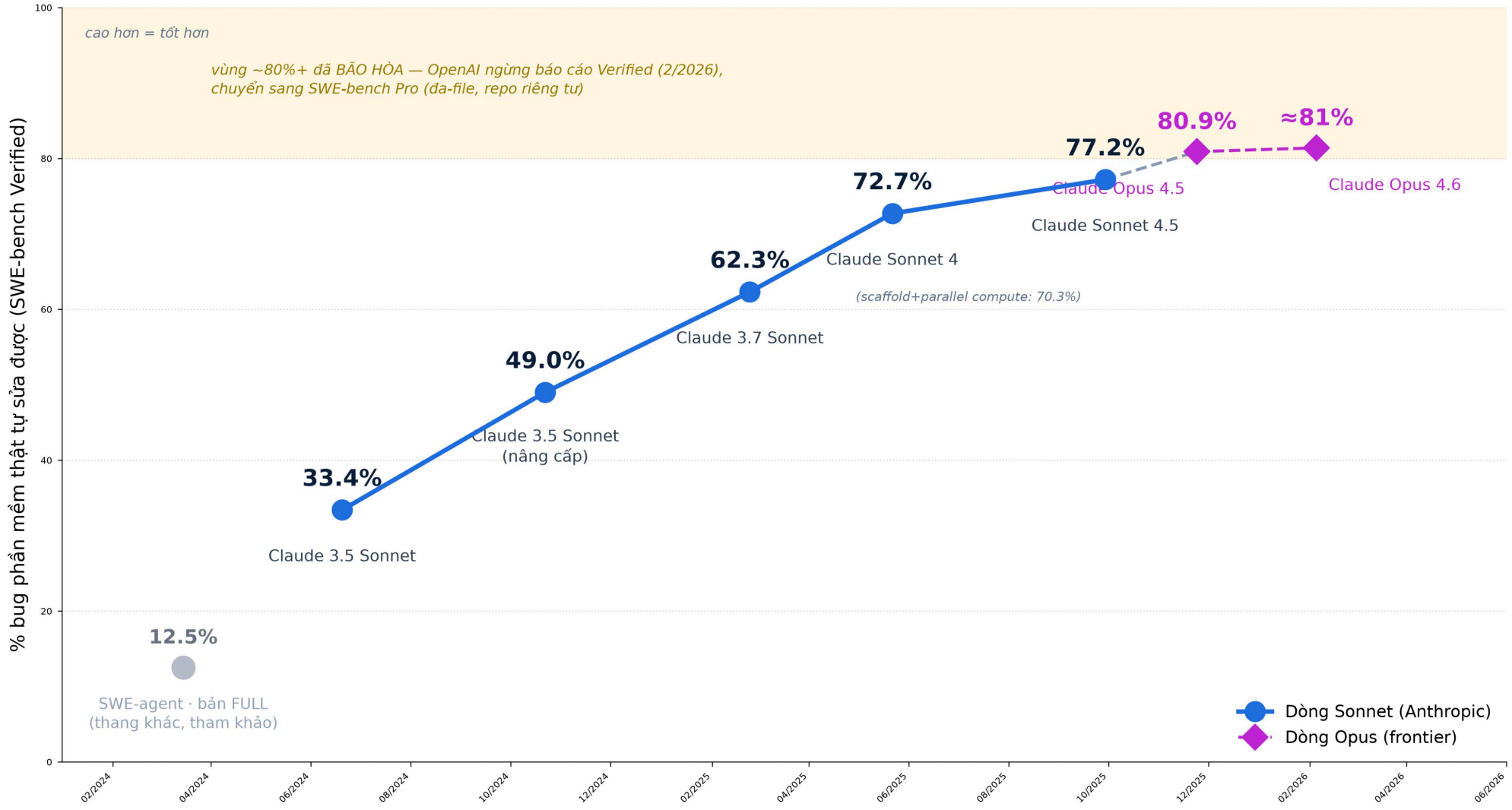
Từ model đơn lẻ sang hệ thống biết hành động



Tập model chọn lọc (không phải điều tra toàn bộ). 2026-H2 tính tới 16/7/2026.

*Làn sóng hiện tại không phải "model nào mạnh hơn" — mà là **system nào dùng model khôn hơn***

SWE-bench Verified: từ 33% (6/2024) lên ~81% (2/2026) — rồi chạm trần bão hòa



33% → ~81% chỉ trong 20 tháng — và đang chạm trần bão hòa quanh ~80%: benchmark này sắp "hết khó" để phân biệt model

Nguồn: SWE-bench Verified = 500 issue GitHub thật, con người đã lọc · điểm công bố chính thức bởi Anthropic (pass@1) · swebench.com — 33.4% (6/2024) → 49.0% (10/2024) → 62.3% → 72.7% → 77.2% → 80.9% → ≈81% (2/2026)

Kiến trúc từ GPT-3 đến nay: cỗ máy vẫn vậy, cách nuôi đã đổi

Lõi Transformer không đổi từ 2017 — như động cơ đốt trong: piston vẫn là piston, nhưng mọi thứ xung quanh được tối ưu điên cuồng.

Cái gì ĐI LÊN

- 👁️ **Cách đánh số ghế khôn hơn** (RoPE) — model nhớ được câu dài mà không lẫn.
- 📓 **Cuốn sổ ghi chú dùng chung** (GQA/MLA) — đọc context dài rẻ đi nhiều lần.
- 🏥 **Bệnh viện đa khoa** (MoE) — 175 tỷ → 2.800 tỷ tham số, mỗi ca chỉ gọi vài chuyên gia.
- 📚 **Bàn làm việc** — từ 2–3 trang (2K) tới 4–5 cuốn sách (1M token).

Cái gì CHẠM TRẦN → trận đua chuyển hướng

- 📕 **Đọc hết sách trong thư viện** (~2024): model đã đọc gần hết văn bản công khai của nhân loại ("data wall") → "to hơn + đọc nhiều hơn" không còn thắng chắc.
- ✍️ **Trận đua mới ① — luyện đề tự chấm** (RLVR): toán có đáp số, code có test → model biết suy luận.
- 🧠 **Trận đua mới ② — được nghĩ kỹ** (test-time compute): cùng một model, cho nháp và thời gian thì khôn hơn hẳn.

Cuộc cách mạng không phải thay động cơ — mà là: nén dữ liệu hiệu quả hơn · luyện bằng bài tập tự chấm · cho model thời gian để nghĩ

Tìm hiểu thêm: RoPE · GQA/MLA · MoE · RLVR · test-time compute — knightli.com — LLM Architecture Evolution 2023–2026 · S. Raschka — The Big LLM Architecture Comparison

Cuộc đua hiện tại (7/2026): ba câu chuyện đáng nhớ

Claude Fable 5 — mạnh nhất, nhưng bị khóa

Anthropic ra model tầng mới (9/6/2026), vượt mọi benchmark — **3 ngày sau bị Mỹ export-control tạm khóa toàn cầu**; bản không giới hạn chỉ cấp cho đội cyberdefense. Model khả dụng mạnh nhất hiện là Opus 4.8.

→ Bài học: phụ thuộc một nhà cung cấp là một rủi ro.

GPT-5.6 — tự chia tầng cho bạn

OpenAI (26/6/2026) ra 3 tầng rõ rệt: **Sol** (flagship, reasoning tối đa), **Terra** (ngang GPT-5.5, rẻ một nửa), **Luna** (nhanh-rẻ).

→ Bài học: chính vendor cũng đang dạy mình "chọn tầng theo việc" — đúng framework ở slide sau.

Kimi K3 — model mở ngang frontier

Moonshot (16/7/2026): 2.800 tỷ tham số MoE, context 1M, **open-weight**, giá chỉ $3/$15 — lần đầu một model mở chơi ngang tốp đầu. Báo chí gọi "cú sốc DeepSeek mới", nhu cầu quá tải cả GPU.

→ Bài học: mở đã bắt kịp đóng thật — self-host không còn là chơi riêng.

*Bản đồ này sẽ cũ trong vài tháng — **thứ bền là cách đọc bản đồ: ai mạnh, ai rẻ, ai mở, ai bị khóa***

Tính đến tháng 7/2026 · Fable 5 · GPT-5.6 · Kimi K3

Từ language model đến multimodal: "token" không chỉ là chữ

Mọi thứ bạn vừa học — token, context, attention — không chỉ dùng cho chữ viết.

Hãy nhớ lại "bàn làm việc" của model: ngày xưa nó chỉ bày được chữ. Giờ người ta **cắt ảnh thành những mảnh nhỏ, cắt tiếng thành những đoạn ngắn** — rồi gọi chúng là "token" y như mảnh chữ, và bày lên đúng cái bàn đó.

Bộ não bên trong không đổi — vẫn là cỗ máy đoán token tiếp theo. Chỉ khác là giờ nó "nhìn" được hình, "nghe" được tiếng: nên model hôm nay (Fable 5, Kimi K3, Gemini) đọc được ảnh, PDF có biểu đồ, audio, cả video.

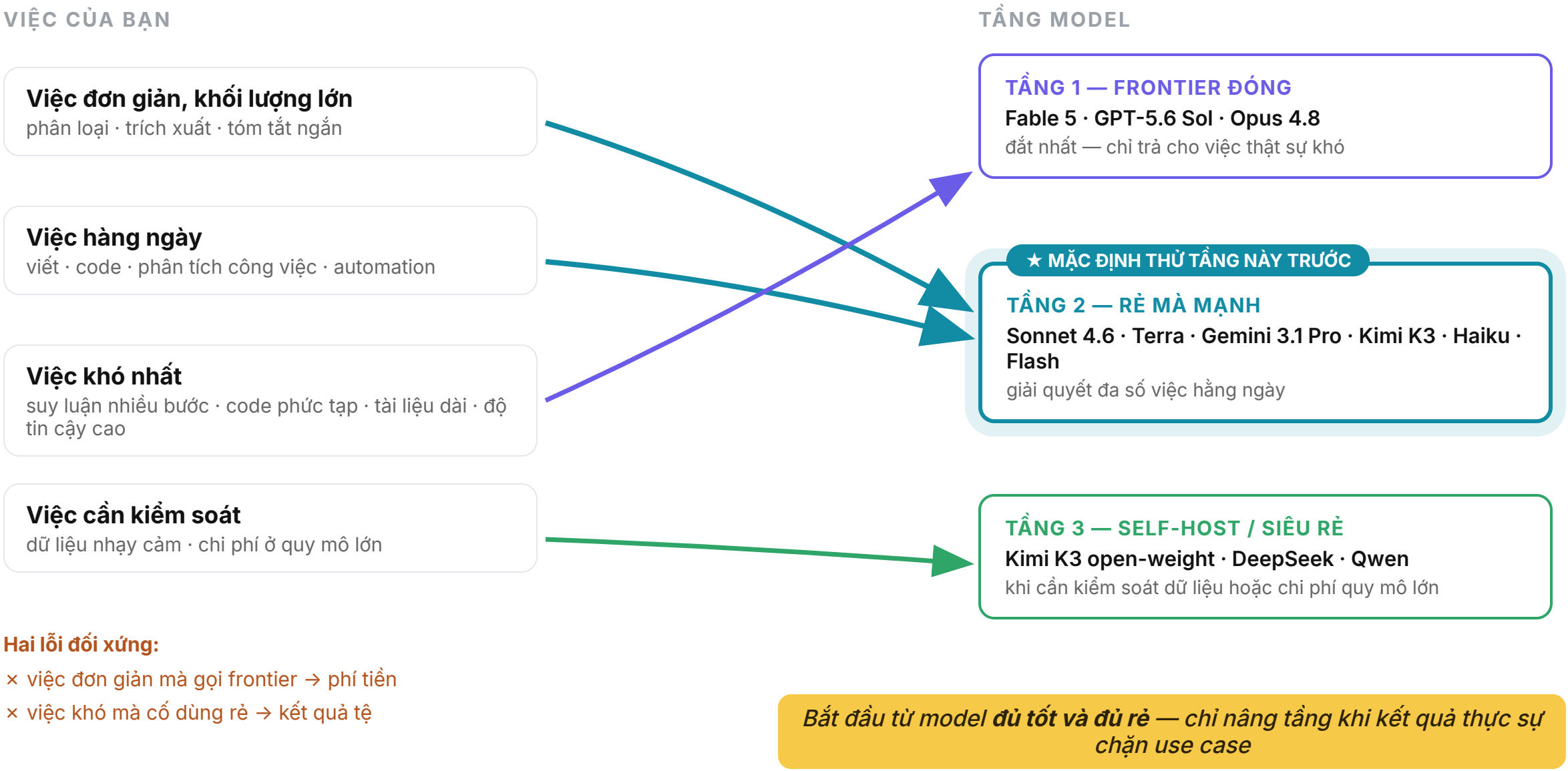
Cùng một cỗ máy đoán-token — đồ đầu vào đã vượt ra ngoài văn bản

PHẦN 06

Chọn model & chi phí token

framework chọn tầng và token economy

Chọn model theo TẦNG, không chọn theo tên



Hai lỗi đối xứng:

× việc đơn giản mà gọi frontier → phí tiền

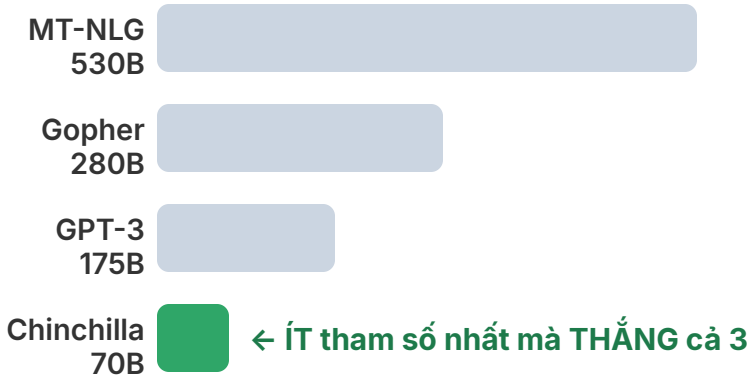
× việc khó mà cố dùng rẻ → kết quả tệ

*Bắt đầu từ model **đủ tốt và đủ rẻ** — chỉ nâng tầng khi kết quả thực sự chặn use case*

Ba trục làm model "giỏi hơn" — tham số chỉ là MỘT trong ba

Trục 1 — Pretraining scale

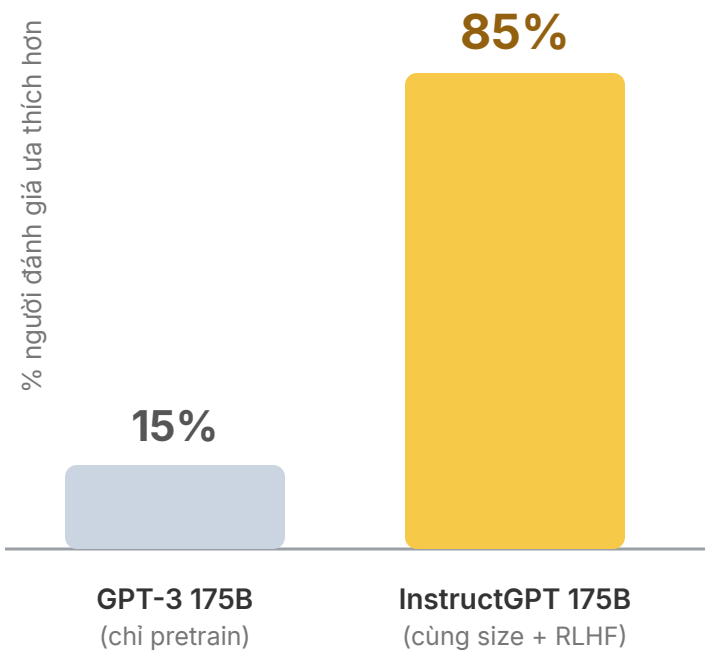
Cùng ngân sách tính toán (Chinchilla, 2022): model nào thắng?



Vì được nuôi bằng **dữ liệu tương xứng đúng tỉ lệ** — **to không bằng cân đối.**

Trục 2 — Post-training

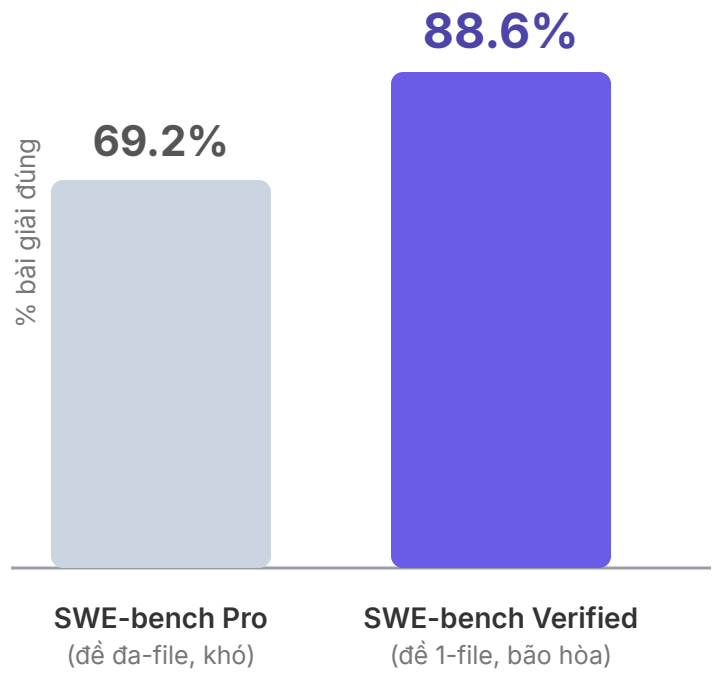
CÙNG 175 tỷ tham số, chỉ khác: có RLHF hay không (InstructGPT, 2022)



Cùng một bộ não — **chỉ khác cách uốn nắn** mà người dùng ưa hẳn.

Trục 3 — Test-time / agentic compute

CÙNG một model (Claude Opus 4.8) — chỉ đổi bộ đề / harness

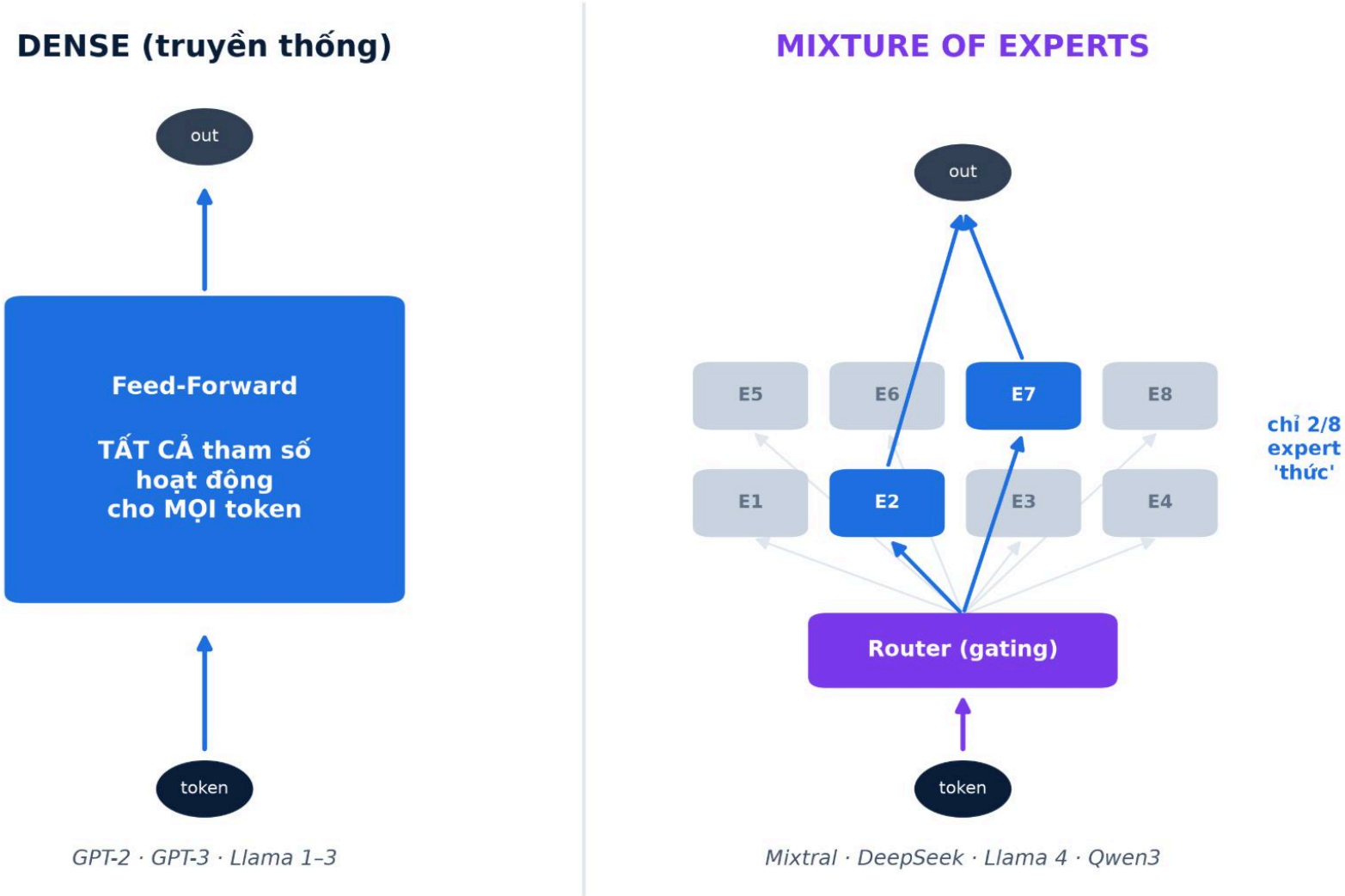


Đổi cách cho model **"được nghĩ kỹ"** (agentic harness) → lệch tới **19 điểm** cùng một model.

Model "giỏi hơn" không chỉ vì to hơn — còn vì cân đối hơn · được uốn nắn hơn · được nghĩ kỹ hơn

Nguồn: Hoffmann et al. 2022 (Chinchilla) · Ouyang et al. 2022 (InstructGPT) · SWE-bench, Claude Opus 4.8 vendor-reported

Mixture of Experts: tăng tham số mà không tăng chi phí tính toán



Mỗi token chỉ đi qua vài "chuyên gia" (ví dụ 2/8) → tổng tham số rất lớn nhưng chi phí mỗi token gần như model nhỏ

Token có giá: vé vào rẻ, vé ra đắt gấp 3–5 lần

VÉ VÀO — INPUT

×1

chữ BẠN gửi đi:
prompt · system instruction · context · lịch sử chat

rẻ — model chỉ cần đọc

VÉ RA — OUTPUT

×3–5

chữ MODEL viết ra — nó phải tự sinh từng mảnh một, vừa chậm vừa tốn

đắt — model phải "vắt óc"

HÓA ĐƠN — 1 LẦN GỌI API

input	1.150 tok × $3 / 1M	**$0.00345**
output	200 tok × $15 / 1M	**$0.00300**
TỔNG		**≈ $0.0065**

số liệu ví dụ — giá thật tùy model & nhà cung cấp

Đọc mục **usage** trong mỗi response — đó là hóa đơn chi tiết giúp bạn kiểm soát chi phí từ ngày đầu.

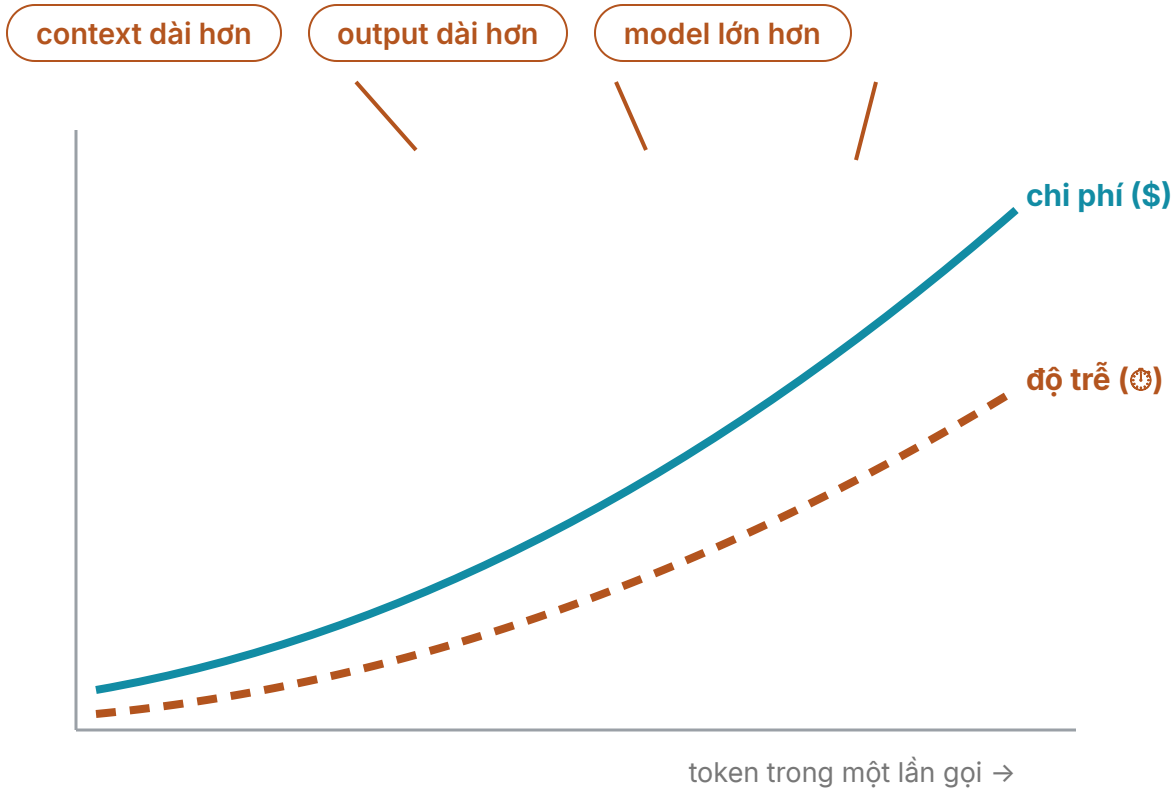
Input tokens + Output tokens = Chi phí mỗi lần gọi — kiểm soát output là núm vặn lớn nhất

Prompt dài = hóa đơn dài — mọi thứ cộng dồn mỗi lần gọi

*Tối ưu chi phí = tối ưu prompt + context — **tóm tắt lại thay vì kéo theo cả lịch sử***

Nhiều token hơn = vừa chậm hơn, vừa đắt hơn

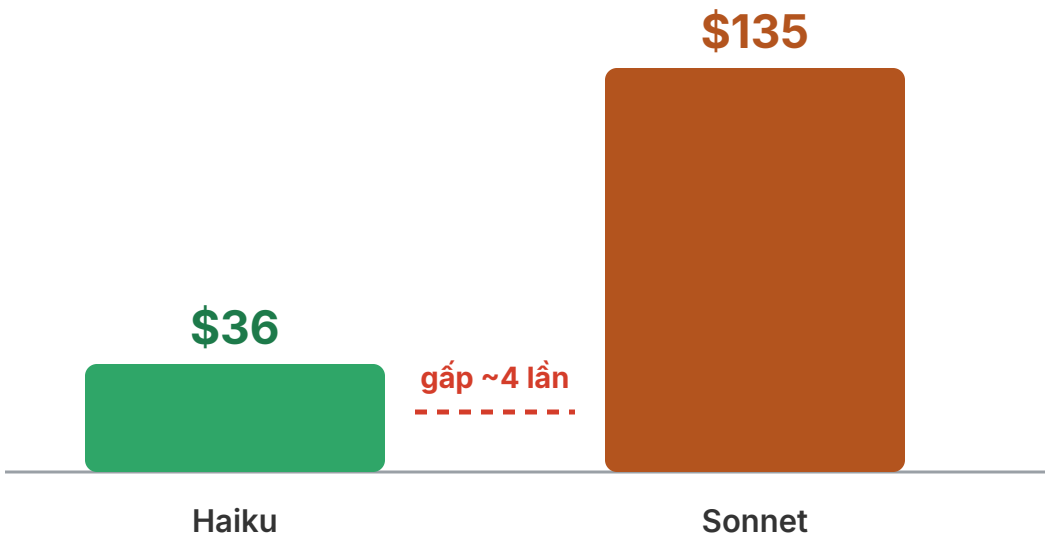
Một núm vặn, hai hệ quả



*Cả hai cùng quy về một thứ: **số token model phải đọc và sinh ra** — đó là "một núm vặn".*

Ví dụ tiền thật — chatbot 1.000 lượt/ngày

`1.350 tok × 1.000 lượt × 30 ngày ≈ 40 triệu token/tháng`



Cùng một việc **đủ tốt**, giá 3/2026 — chọn sai tầng là trả đắt gấp ~4 lần **mỗi tháng**. Slide chọn tầng vừa rồi không phải lý thuyết — nó là tiền thật.

*Cái gì làm chậm cũng làm đắt → **tối ưu cả hai cùng lúc bằng cách tiết kiệm token***

Cùng một prompt — ba model, ba phong cách trả lời

💬 Prompt: *"Tóm tắt báo cáo tài chính Q1 trong 3 bullet và nêu 1 rủi ro chính."*

Claude — consulting style

Tóm tắt Q1:
1. Doanh thu: +12% nhờ kênh online
2. Chi phí vận hành: −8% sau tinh gọn
3. Dòng tiền: cải thiện, đủ 6 tháng vận hành

Rủi ro chính: tồn kho tăng 20% — cần phương án xử lý trong Q2.

Mạch lạc, thiên cấu trúc → **hợp phân tích, viết tài liệu dài**

GPT — ngắn gọn · tự nhiên

- Q1 khá ổn: doanh thu +12%, chi phí −8%, dòng tiền dương.
- Điểm sáng lớn nhất là kênh online.
- Rủi ro chính: tồn kho +20%, nên xả bớt trong Q2.

Nói gọn: ổn — nhưng coi chừng kho hàng. 👍

Tự nhiên, linh hoạt → **hợp app/chat đa dụng, hệ sinh thái lớn**

Gemini / Kimi — mạnh context dài

Đối chiếu **40 trang báo cáo + 3 file đính kèm**:
- DT +12%; online chiếm 61% tổng DT
- Chi phí −8% nhờ tinh gọn 2 kho
- Dòng tiền dương, đủ 6 tháng

Rủi ro chính: tồn kho +20% — vượt ngưỡng an toàn *(mục 7.2)*.

Bám nhiều tài liệu → **hợp workflow nhiều file, cửa sổ 1M token**

Chọn model không chỉ là chọn giá và điểm số — còn là chọn phong cách

Bài tập về nhà: lấy một prompt trong công việc của bạn, chạy thử trên 2–3 model, so sánh. Phong cách thay đổi theo thế hệ model.

Benchmark có đáng tin không? — tin vừa thôi

1 Model học vẹt đường tắt

Điểm cao có thể nhờ ăn gian dữ liệu (spurious cues) — như slide "học vẹt" vừa rồi.

2 Đề thi bị bão hòa

SWE-bench Verified: 33% → ~81% trong 20 tháng → sắp "hết khó" để phân biệt model, phải ra đề mới (SWE-bench Pro).

3 Học tủ đề (benchmaxxing)

Model có thể được luyện đúng dạng đề để ăn điểm — điểm tăng không hẳn năng lực tăng.

Ví dụ profile không phẳng (2023): GPT-4 đỗ **Bar exam** (kỳ thi luật sư Mỹ) ở top 10% — nhưng **Codeforces** (thi lập trình thi đấu) dưới 5%. Điểm cao ở kỳ thi này không nói gì về kỳ thi khác.

*Benchmark là **tín hiệu, không phải bằng chứng**. Chỉ có một bài test đáng tin hoàn toàn: **việc của chính bạn, trên dữ liệu của chính bạn**.*

Nguồn: swebench.com · Zhong et al. (2022), ICML · Stanford AI Index.

PHẦN 07

Gọi API lần đầu

điều khiển một vòng next-token từ xa

Một lần gọi API diễn ra thế nào?

① Prompt
system + user + context

→

② API call
gửi request tới provider

→

③ Token stream
model sinh từng mảnh

→

④ Response
nội dung + usage + lý do dừng

Gọi API = điều khiển một vòng next-token từ xa — không phép màu, đúng cơ chế mình vừa học

Mỗi API call luôn có 3 thứ phải kiểm soát cùng lúc: **chất lượng — độ trễ — chi phí**.

Giải phẫu một prompt: bốn lớp xếp chồng

1 PROMPT = 4 PHẦN

LỚP 1
System instruction

"Lời dặn đầu ca": model là ai, cư xử thế nào, không được làm gì

`«Bạn là trợ lý y khoa, trả lời ngắn gọn, không chẩn đoán…»`

LỚP 2
User input

Câu hỏi / yêu cầu của người dùng trong lượt này

`«Tóm tắt báo cáo Q1 giúp mình»`

LỚP 3
Context bổ sung

Tài liệu, lịch sử chat, dữ liệu tra sổ — phần bày lên "bàn làm việc"

`«[đính kèm: bao_cao_q1.pdf — 3 đoạn liên quan]»`

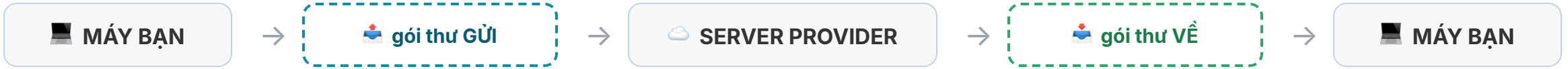
LỚP 4
Output mong muốn

Dạng kết quả: gạch đầu dòng? bảng? JSON? dài bao nhiêu?

`«3 bullet + 1 rủi ro chính, tiếng Việt»`

*Viết rõ cả 4 lớp = đã làm tốt một nửa "prompt engineering" — **phần còn lại là các ngày sau***

Giải phẫu một API call: gói thư gửi và gói thư về



REQUEST — gói thư gửi đi

```
POST api.openai.com/v1/chat/completions
{
  "model": "gpt-5.6-terra", (1)
  "messages": [ (2)
    { "role": "system", "content": "Bạn là trợ lý tài chính, trả lời ngắn gọn." },
    { "role": "user",   "content": "Tóm tắt báo cáo Q1: 3 bullet + 1 rủi ro." }
  ],
  "max_tokens": 500, (3)
  "temperature": 0 (4)
}
```

(1) tên model — "số tổng đài" · (2) 3 vai trò: system / user / assistant
(3) trần độ dài trả lời · (4) độ "liều" (0 = ổn định)

RESPONSE — gói thư nhận về

```
{
  "choices": [{
    "message": { "role": "assistant",
      "content": "• Doanh thu Q1 +12%…\n• Chi phí −8%…\n• Rủi ro: tồn kho +20%." }, (5)
    "finish_reason": "stop" (6)
  }],
  "usage": { (7)
    "prompt_tokens": 1150,  // vé vào
    "completion_tokens": 200,  // vé ra
    "total_tokens": 1350
  }
}
```

(5) câu trả lời ở **choices[0].message.content**
(6) stop = tự kết thúc | length = hết hạn mức | tool_calls · (7) hóa đơn chi tiết

Đọc usage mỗi lần gọi — đừng để cuối tháng mới giật mình nhìn hóa đơn

platform.openai.com/docs · docs.anthropic.com

Hai núm vặn chọn từ: temperature & top_p

temperature — "núm vặn độ liều"

Cùng một câu: *"Một tách ___"* — bảng xác suất đổi theo T

T = 0

luôn chọn từ **chắc nhất** → ổn định, lặp lại, hợp code & phân tích

T = 1

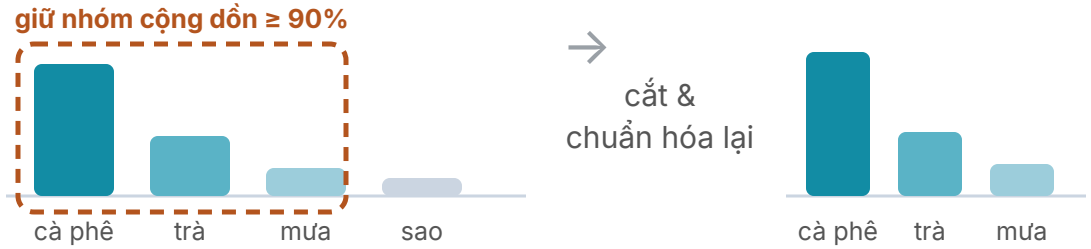
cân bằng tự nhiên — vẫn ưu tiên từ hợp lý

T = 2

phân bố **phẳng ra** → đa dạng, "phiêu", dễ lạc đề

top_p — "chỉ xem top đầu bảng" (p = 0.9)

① Bảng xác suất gốc ② Bảng mới



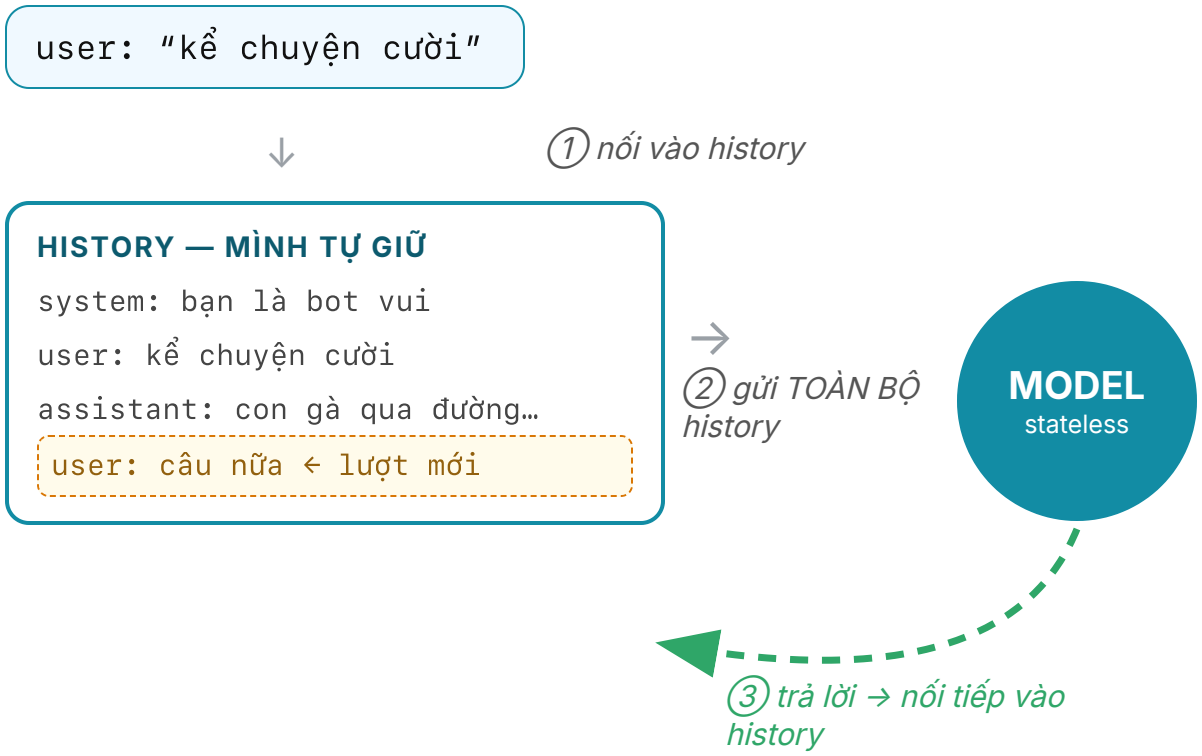
"sao" (đuôi dài xác suất thấp) **bị loại khỏi lựa chọn** — model chỉ còn chọn trong nhóm đáng tin. Thường chỉ vặn **một trong hai**: temperature *hoặc* top_p.

> **Lưu ý quan trọng:** hai núm này không làm model thông minh hơn — chỉ đổi **cách chọn từ**, không thêm tri thức.

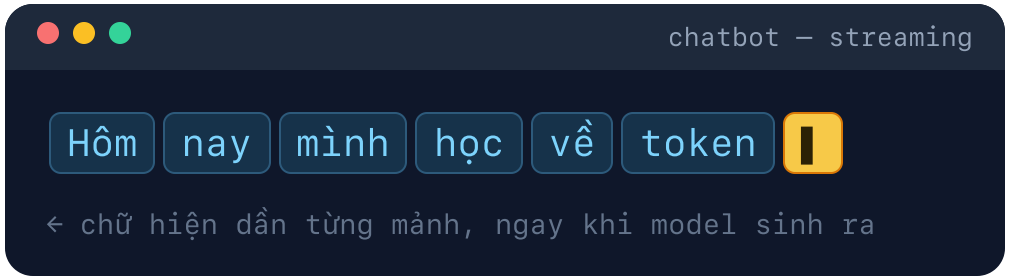
*Mặc định an toàn: **temperature = 0** cho việc cần ổn định — chỉ tăng khi thật sự cần đa dạng*

Chatbot = vòng lặp + trí nhớ; streaming = nhả chữ từng mảnh

"Trí nhớ" của chatbot đến từ đâu?



Streaming — next-token nhìn tận mắt



Đây chính là bản chất **next-token**: model đoán → nhả một mảnh → đoán tiếp. Giao diện "đang gõ" chỉ là **lộ trình của vòng lặp**.

Model không nhớ gì giữa hai lần gọi — "trí nhớ" là do MÌNH gửi lại history mỗi lần

Hai "số tổng đài" lớn — và khi nào tự nuôi model tại nhà

OpenAI vs Anthropic — cú pháp tương đương

Cùng một logic: gửi messages, nhận content + usage. Khác tên hàm và cách bóc kết quả:

```
OpenAI: client.chat.completions.create(...) →
.choices[0].message.content
Anthropic: client.messages.create(...) →
.content[0].text
```

Self-host (open-weight)

Tải "bộ não" mở (Kimi K3, Qwen, Llama) về chạy trên máy mình:

✓ dữ liệu không rời khỏi tay bạn

✓ không trả tiền theo token

× tự lo GPU, vận hành, cập nhật

Đổi **base_url** (số tổng đài) là code gọi API chuyển sang model tự host gần như nguyên vẹn.

*API không chỉ là cách gọi model — đó là **mức quyền truy cập** bạn có với model đó*

PHẦN 08

Tổng kết

những ý để mang về

Key takeaways — 5 ý để mang về

1. LLM = cỗ máy Transformer **đoán token tiếp theo từ context** — mọi thứ khác là hệ quả.

2. Từ cỗ máy đoán chữ thành trợ lý: pre-training → SFT → căn chỉnh → **luyện đề tự chấm & được nghĩ kỹ**.

3. Model có giới hạn bẩm sinh: bong bóng thời gian, nói chắc như đúng rồi, bàn làm việc có hạn — nên **đừng tin benchmark, hãy tự test**.

4. Chọn model theo **tầng theo việc**, kiểm soát 3 núm: **chất lượng — độ trễ — chi phí**.

5. Gọi API là **điều khiển một vòng next-token từ xa** — kèm một mức quyền truy cập nhất định vào model.

TRẢ LỜI CÂU HỎI ĐẦU NGÀY

"Bên trong AI đang làm gì?"

— một vòng lặp đoán token, được nuôi bằng dữ liệu, đang chờ bạn điều khiển.

Buổi chiều nay, bạn sẽ trả lời câu hỏi đó bằng hành động: **gọi API đầu tiên và build chatbot của chính mình**.

Một lời nhắc nhỏ mang theo: dữ liệu là mạch sống của model nhưng cũng là phần kém minh bạch nhất. Model nền là điểm đòn bẩy lớn — và cũng có thể là điểm lỗi lan xuống mọi ứng dụng. Evaluation, guardrails và system design không bao giờ là phần phụ.

Sáng nay bạn đã hiểu AI đang làm gì. Chiều nay — điều khiển nó bằng chính tay bạn.

Appendix — xem & đọc thêm sau buổi học

🎬 Nên xem & chơi trước (khuyên bắt đầu từ đây)

3Blue1Brown — Transformers, the tech behind LLMs · video giải thích Transformer bằng hình động dễ hiểu nhất hiện nay — youtube.com/watch?v=wjZofJX0v4M

3Blue1Brown — Attention in transformers, step-by-step · phần tiếp theo, đi sâu vào attention — youtube.com/watch?v=eMlx5fFNoYc

Transformer Explainer · chạy GPT-2 ngay trong trình duyệt: tự chỉnh temperature, xem next-token probs và attention map — poloclub.github.io/transformer-explainer

Karpathy — nanoGPT & State of GPT · người giải thích lại mọi thứ này bằng code chạy được — github.com/karpathy/nanoGPT · youtube

📄 Paper nền tảng

Attention Is All You Need (Vaswani et al., 2017) · paper khai sinh Transformer — chữ T trong GPT — arxiv.org/abs/1706.03762

InstructGPT (Ouyang et al., 2022) · vì sao ChatGPT biết nghe lời — arxiv.org/abs/2203.02155 · **DPO** (2023) — arxiv.org/abs/2305.18290

Emergent World Representations (Li et al., ICLR 2023) · Othello-GPT — bằng chứng model tự xây world model — arxiv.org/abs/2210.13382 · bản đọc dễ hơn: The Gradient

On the Dangers of Stochastic Parrots (Bender et al., FAccT 2021) · phía phản biện nổi tiếng — doi.org/10.1145/3442188.3445922

🔬 Đào sâu thêm

Probe & can thiệp Othello-GPT — Nanda et al. 2023 · **Diffusion model "thấy" thế giới từ step 1** — arxiv.org/abs/2306.05720 · **Dynamometer car** — Viégas & Wattenberg 2023

Dùng LLM để hiểu LLM (Steinhardt, BAIR) — arxiv.org/abs/2302.14233 · bài nói · **Gán nhãn attention head CLIP** — arxiv.org/abs/2310.05916 · **Representation Engineering / ITI** — arxiv.org/abs/2306.03341 · arxiv.org/abs/2310.01405

📊 Số liệu & bảng giá (7/2026)

Bảng model & giá (7/2026): Fable 5 $10/$50 (tạm khóa export-control) · GPT-5.6 Sol/Terra/Luna · Opus 4.8 $5/$25 · Sonnet 4.6 $3/$15 · Haiku 4.5 $0.8/$4 · Gemini 3.1 Pro $2/$12 · Kimi K3 $3/$15 open-weight · DeepSeek V3 (API siêu rẻ)

Stanford AI Index — hai.stanford.edu/ai-index · **SWE-bench** — swebench.com · **Giá API** — openai.com/api/pricing · anthropic.com/pricing · **AI 2027** — ai-2027.com *(kịch bản gây tranh luận — đọc phản biện)*